Distributional Learning of Context-Free Languages under Fixed Finite-Monoid Typing

Takayuki Kuriyama
Independent Researcher
growup.kuriyama@gmail.com

Abstract

We study positive-data learning of context-free languages under a fixed homomorphism $h : \Sigma^* \to M$ into a finite monoid. A language is $\sim_h$-substitutable if nonempty factors of the same h-type that share one context have identical sets of contexts. For each fixed h, we define a set-driven reconstruction operator that recovers every context-free $\sim_h$-substitutable target from finitely many positive witnesses, and a separate conservative sequential learner that identifies every nonempty target in the limit; reconstruction and updates take polynomial time. Our characteristic-data bound depends on shortest yields in a target grammar refined by h-values, which can be exponentially larger than ordinary grammar thickness in some presentations. The classical characterization of locally trivial finite semigroups gives an exact criterion: h is determined on nonempty factors by a fixed prefix–suffix window exactly when $h(\Sigma^+)$ is locally trivial. In this regime, and more generally when the positive image is a homomorphic image of a subsemigroup of a product of a locally trivial finite semigroup and a finite group, ordinary grammar thickness suffices. The union of locally trivial slices coincides with Yoshinaka's fixed-window hierarchy. For linear targets we obtain characteristic data polynomial in linear grammar size under every fixed h. We also exhibit nonregular linear and nonlinear context-free targets outside every fixed-window class, while the one-bracket Dyck language lies outside every fixed-typing class.

Keywords: Grammatical inference; Context-free languages; Distributional learning; Positive-data identification; Finite monoids

1 Introduction

1.1 Background and motivation

Gold [3] showed that no language class containing all finite languages and at least one infinite language is identifiable in the limit from positive data alone. For context-free languages, distributional learning recovers positive results by restricting which substrings may be compared across contexts; classical examples are Clark–Eyraud substitutability [9] and Yoshinaka's (k, ℓ)-substitutability [16]. See de la Higuera [8] for general background and Kanazawa [2, 1] for positive-data learning of categorial grammars. In particular, Kanazawa's monograph develops characteristic samples for rigid grammars and finite-elasticity arguments for the string languages of k-valued categorial grammars [1, Chapters 5 and 8]. This is a complementary line of research: here the learning bias instead fixes a finite-monoid typing of terminal factors and requires distributional substitutability.

Historical priority and general signature learning. This paper is a substantially corrected and extended version of the author's 2014 preprint [11], which introduced substitutability relative to recognizable equivalence relations and proposed positive-data learning in that setting. The 2014 preprint predates the publication of Yoshinaka's general many-sorted signature

account [12]; this chronology concerns the earlier formulation, not the validity of all proofs or quantitative claims in that preliminary version. In Yoshinaka's framework, for the finite-monoid homomorphism $h : \Sigma^* \to M$ considered in this paper, the nonempty fibers $h^{-1}(m) \cap \Sigma^+$ can be represented as disjoint sorts, with typed concatenation and an operation promoting a typed factor to the whole-string sort. The resulting signature has the same substitutability condition on nonempty factors as the fixed-h definition below. Consequently, the *existence* of a positive-data learner for a fixed h is also covered by the qualitative signature-based result of [12, Theorem 1]; our particular batch and sequential constructions are analyzed directly here. The finite-monoid-specific contributions are the algebraic fixed-window criterion and closure-by-retyping results, the *encoded size* bounds for characteristic data (rather than only witness cardinality), and the linear and expressive-boundary results. The broader distributional learning setting is surveyed by Clark and Yoshinaka [13]; local substitutability is studied by Coste, Garet, and Nicolas [14] and Coste and Nicolas [15].

We study a recognizable-congruence version of this program. Let $h : \Sigma^* \to M$ be a homomorphism into a finite monoid and $\sim_h := \ker h$. For $x \in \Sigma^*$, write $\mathcal{D}_L(x) := \{(u, v) : uxv \in L\}$. A language L is *$\sim_h$-substitutable* if, for nonempty x, y, $h(x) = h(y)$ and $\mathcal{D}_L(x) \cap \mathcal{D}_L(y) \neq \emptyset$ imply $\mathcal{D}_L(x) = \mathcal{D}_L(y)$. Write $\mathcal{C}_h^{\mathrm{cf}}$ for the context-free $\sim_h$-substitutable languages, and call this the *fixed-h slice*. Thus full distributional equality is required only after a fixed finite algebraic comparison.

For a minimal example, take $\Sigma = \{a\}$ and $L = \{a, aa\}$. Although $(\lambda, \lambda) \in \mathcal{D}_L(a) \cap \mathcal{D}_L(aa)$, the context (a, λ) belongs to $\mathcal{D}_L(a)$ but not to $\mathcal{D}_L(aa)$. Thus L is not substitutable under the trivial typing. Under the parity morphism $h(a^n) = n \bmod 2$ into $\mathbb{Z}/2\mathbb{Z}$, however, a and aa have distinct types and $L \in \mathcal{C}_h^{\mathrm{cf}}$.

Finite-index monoid congruences, equivalently kernels of morphisms into finite monoids, provide finite typings. For each fixed h, write RS_h for the class of all $\sim_h$-substitutable languages, without a context-freeness restriction. For expressive comparison, let

$$\mathsf{RS} := \bigcup_h \mathsf{RS}_h,$$

where h ranges over finite-monoid homomorphisms on Σ^*. Every regular language belongs to this union. Our learning results, however, concern only $\mathcal{C}_h^{\mathrm{cf}}$ for one fixed h; no learner is claimed for RS.

Unlike one fixed prefix–suffix window, h can combine independent finite-state factor features by a product. If the finite typing is allowed to be refined accordingly, the framework is stable under regular filtering and arbitrary inverse homomorphisms, including erasing ones (Proposition 3.2). This is a change-of-typing statement, not closure of one fixed-h slice. An algebraic criterion identifies the typings determined by fixed prefix–suffix windows on nonempty factors (Proposition 3.5); combining windows with finite groups yields a broader characteristic-data bound (Corollary 7.4). Examples below establish nonregular linear and nonlinear reach beyond fixed windows, and counter/Dyck obstructions bound that reach.

Specifying h is a prior choice, related to typing or domain bias in automata inference [20]; here types apply to entire terminal factors, not to occurrences with their outer contexts. For linear grammars, yield typing also induces a regular control set of production-label sequences, giving a limited but concrete connection with Takada's regular control-set approach [17, 18] (Section 8), without an equivalence claim.

1.2 Contributions and main results

The paper has three main contributions.

(1) *Learning for fixed h.* A set-driven reconstruction operator $\mathcal{B}_h$ reconstructs every target in $\mathcal{C}_h^{\mathrm{cf}}$ exactly from finitely many canonical positive witnesses, and a separate conservative

sequential learner $\mathcal{A}_h$ identifies every nonempty target in Gold's sense; reconstruction and updates take time polynomial in the accumulated sample.

(2) *Quantitative guarantees.* For arbitrary fixed h, we bound the characteristic data by grammar size and by the shortest terminal yields after target nonterminals are split according to their h-values. When the finite typing is refined by a fixed prefix–suffix window and a finite-group value, ordinary source-grammar thickness suffices. Linear fixed-h targets admit characteristic data polynomial in the size of a linear CFG. We also show that the shortest yields after splitting can be exponentially longer than those in the source grammar.

(3) *Structural and expressive position.* We establish closure properties obtained by finite refinement and product of typings (rather than closure of a single fixed-h slice), characterize finite typings determined by fixed prefix–suffix windows, and relate linear yield typing to regular control sets. Explicit nonregular linear and nonlinear examples lie beyond fixed windows; counter/Dyck obstructions and comparison with Clark's congruential family delimit the framework.

1.3 Organization of the paper

Sections 2–3 set up the framework, Sections 4–5 prove learnability, Sections 6–8 give the quantitative results, and Section 9 gives the expressiveness comparisons. Long normalization proofs are deferred to the appendices.

2 Preliminaries

Let Σ be a finite alphabet, let Σ^* be the free monoid with identity λ, and put $\Sigma^+ := \Sigma^* \setminus \{\lambda\}$. A pair $(u, v) \in \Sigma^* \times \Sigma^*$ is called a *terminal context*. For $L \subseteq \Sigma^*$ and $x \in \Sigma^*$, the *distribution* of x in L is $\mathcal{D}_L(x) := \{(u, v) \in \Sigma^* \times \Sigma^* : uxv \in L\}$. The *syntactic congruence* of L is defined by $x \equiv_L y$ iff $\mathcal{D}_L(x) = \mathcal{D}_L(y)$, for $x, y \in \Sigma^*$ [5, Chapter III, Proposition 10.2]. We say that x and y are *syntactically congruent* when $x \equiv_L y$. As usual, $\equiv_L$ is a monoid congruence: if $x \equiv_L x'$ and $y \equiv_L y'$, then $xy \equiv_L x'y'$.

For a word $w \in \Sigma^*$ and a letter $a \in \Sigma$, let $|w|_a$ denote the number of occurrences of a in w.

A context-free grammar (CFG) is a tuple $G = (V, \Sigma, P, S)$ with finite nonterminal set V, start symbol $S \in V$, and productions $A \to \alpha$ with $A \in V$ and $\alpha \in (V \cup \Sigma)^*$. A *sentential form* is any word in $(V \cup \Sigma)^*$. We write $\alpha \Rightarrow_G \beta$ for one derivation step in G, and $\Rightarrow_G^*$ for its reflexive-transitive closure. Thus $L(G) := \{w \in \Sigma^* : S \Rightarrow_G^* w\}$. For $A \in V$, put $L_G(A) := \{w \in \Sigma^* : A \Rightarrow_G^* w\}$. For a nonterminal $X \in V$, a terminal context (u, v) is a *terminal reaching context of X in G* if $S \Rightarrow_G^* uXv$. A nonterminal is *reachable* if it occurs in a sentential form reachable from S, and *productive* if it derives a terminal word. A nonterminal is *useless* if it is not both reachable and productive, and it is *nullable* if it derives λ. A production $A \to B$ with $A, B \in V$ is a *unit production* (or *unit rule*). The *unit closure* is the reflexive-transitive closure of the relation on nonterminals induced by unit productions. A grammar is *reduced* if every nonterminal is reachable and productive. For a grammar generating a nonempty language, its *trim* (or *reduced part*) is obtained by first deleting all nonproductive nonterminals and their incident productions, then deleting from the remaining grammar all nonterminals unreachable from the start symbol, together with their incident productions. A CFG is *linear* if every right-hand side contains at most one nonterminal. By *binarization* we mean replacement of longer right-hand sides by equivalent chains of binary productions using fresh nonterminals. We write CFL for the class of context-free languages. For any finite representation $\mathcal{R}$, $|\mathcal{R}|$ denotes the length of a fixed reasonable encoding; for a grammar this is polynomially equivalent to the usual symbol count of its productions. Following the thickness parameter used by Wakatsuki–Tomita [10] and adopted by Yoshinaka [16] (see also Eyraud–Heinz–Yoshinaka [22, Definition 2.11]), for

a reduced CFG $G = (V, \Sigma, P, S)$ we use the shortest-yield *thickness*

$$\tau_G := \max_{A \in V} \min\{\, |w| : A \Rightarrow_G^* w,\ w \in \Sigma^* \,\}.$$

2.1 Learning from positive data

We use Gold's explanatory model [3]. Let H_0 denote a fixed CFG with one start symbol and no productions, so $L(H_0) = \emptyset$. We use texts only for nonempty targets and represent the empty language separately by H_0; equivalently one may adjoin the usual pause symbol. For a nonempty language L, a *text* is an infinite sequence $T = (w_1, w_2, \ldots)$ over L in which every word of L occurs. Write $T[n] = (w_1, \ldots, w_n)$ and $K_n := \{w_1, \ldots, w_n\}$. A learner $\mathcal{A}$ *identifies a nonempty L in the limit* if for every text T for L there are n_0 and a grammar H such that $\mathcal{A}(T[n]) = H$ for every $n \geq n_0$ and $L(H) = L$. Thus the hypothesis representation itself eventually stabilizes. A sequential learner is *conservative* if it changes its current hypothesis only when the newly presented positive datum is not generated by that hypothesis.

We distinguish this sequential learner from a *set-driven reconstruction operator* $\mathcal{B}$: this is a map from finite sets of words to grammars; in particular, its output is independent of presentation order and multiplicity. We call $\mathcal{B}$ *sample-consistent* if $K \subseteq L(\mathcal{B}(K))$ for every finite sample K. A finite set $C \subseteq L$ is a *characteristic sample of L for $\mathcal{B}$* if $C \subseteq K \subseteq L$ implies $L(\mathcal{B}(K)) = L$ for every finite K. This is the positive-data specialization of the characteristic-sample component of de la Higuera's framework [7]; see also [8, Section 7.3.3, Definition 7.3.3] for learner-relative characteristic samples and their encoded-size convention. The latter is formulated for sequential presentations, whereas our condition is explicitly set-driven. We measure sample size by $\|K\| := \sum_{w \in K}(|w| + 1)$, so λ contributes one unit.[1] We use *characteristic data* only as shorthand for the encoded size $\|C\|$ of a characteristic sample C; the quantitative results below combine polynomial-time reconstruction with polynomial characteristic-data size. Polynomial update time and polynomial characteristic-sample size are distinct criteria in [8, Sections 7.3.2–7.3.3]; neither alone establishes incremental polynomial time-and-data identification. Eyraud–Heinz–Yoshinaka [22, Definitions 2.5 and 2.8] distinguish incremental polynomial time-and-data identification from its set-based counterpart. The incremental criterion requires a correct and stable hypothesis as soon as a characteristic sample has appeared in the presentation; the set-based criterion requires correct reconstruction from any finite sample containing a characteristic sample. Our polynomial characteristic-data guarantees concern the set-driven operator $\mathcal{B}_h$, not the incremental polynomial-data behavior of $\mathcal{A}_h$. Gold stabilization of the latter is established separately in Corollary 5.5.

2.2 Fixed finite-monoid typing

Fix a finite monoid M and a homomorphism $h : \Sigma^* \to M$, represented by the multiplication table of M and the letter images $h(a)$ for $a \in \Sigma$. Thus $h(w)$ is computable by a left-to-right product in $O(|w|)$ time; this is the sense in which the fixed homomorphism is *efficiently computable* below.

For a finite semigroup S, let $E(S)$ be its idempotents. We call S *locally trivial* if $ete = e$ for all $e \in E(S)$ and $t \in S$. The *positive image semigroup* of h is $h(\Sigma^+)$; it need not contain the identity of M.

Definition 2.1 (fixed-h substitutability)**.** *A language $L \subseteq \Sigma^*$ is* $\sim_h$-substitutable *if for all $x, y \in \Sigma^+$, equality $h(x) = h(y)$ together with $\mathcal{D}_L(x) \cap \mathcal{D}_L(y) \neq \varnothing$ implies $\mathcal{D}_L(x) = \mathcal{D}_L(y)$.*

The empty word is not used as an internal substitutable fragment. When $\lambda \in L$, the reconstruction handles it separately through the start symbol.

[1] The $+1$ prevents a nonempty sample such as $K = \{\lambda\}$ from having size 0; it also gives $|K| \leq \|K\|$, so the number of examples is controlled by the same measure.

For a nonempty internal factor x, we call $h(x)$ its *yield type*. As above, $\mathcal{C}_h^{\mathrm{cf}}$ denotes the context-free $\sim_h$-substitutable languages, and we define $\mathcal{C}_h^{\mathrm{lin}} := \{L \in \mathcal{C}_h^{\mathrm{cf}} : L \text{ is generated by some linear CFG}\}$.

3 Fixed Recognizable Substitutability

For fixed h, put $\mathsf{RS}_h := \{L \subseteq \Sigma^* : L \text{ is } \sim_h\text{-substitutable}\}$. Then the fixed-$h$ slice is $\mathcal{C}_h^{\mathrm{cf}} = \mathsf{RS}_h \cap \mathrm{CFL}$, while $\mathcal{C}_h^{\mathrm{lin}}$ consists of its linearly generated members. For expressive comparisons only, let $\mathsf{RS} := \bigcup_h \mathsf{RS}_h$, where h ranges over finite-monoid homomorphisms on Σ^*. We write $\mathsf{RS}_h(\Gamma)$ and $\mathsf{RS}(\Gamma)$ when the alphabet matters; all inclusions below are alphabetwise. The learning results always concern one fixed h.

Proposition 3.1. *A language is regular iff it has the form $h^{-1}(\mathsf{Acc})$ for some homomorphism $h : \Sigma^* \to M$ into a finite monoid and some $\mathsf{Acc} \subseteq M$. For regular L, h may be taken to be its syntactic morphism, whose kernel is $\equiv_L$. For every fixed h and $\mathsf{Acc} \subseteq M$, the language $h^{-1}(\mathsf{Acc})$ is $\sim_h$-substitutable. Thus every regular language belongs to RS.*

Proof. The first two statements follow from the classical syntactic-monoid characterization [5, Chapter III, Propositions 10.1–10.2]. If $L = h^{-1}(\mathsf{Acc})$ and $h(x) = h(y)$, then $h(uxv) = h(uyv)$ for all u, v, so $\mathcal{D}_L(x) = \mathcal{D}_L(y)$. □

By Proposition 3.1, RS and $\mathsf{RS} \cap \mathrm{CFL}$ contain every finite language and Σ^*; Gold's theorem makes both unions nonlearnable from positive data on nonempty alphabets. Yoshinaka [16, Section 4] notes the analogous issue for the union of fixed-window classes. We learn only fixed-h slices.

Typing refinement is monotone on the nonempty factors that enter Definition 2.1. If $\ker h'|_{\Sigma^+} \subseteq \ker h|_{\Sigma^+}$, then $\mathsf{RS}_h \subseteq \mathsf{RS}_{h'}$, since h' requires fewer factor comparisons. Refinement may nevertheless enlarge the typed grammar used in the quantitative analysis. For two finite typings h, g, $\ker(h \times g)|_{\Sigma^+} = \ker h|_{\Sigma^+} \cap \ker g|_{\Sigma^+}$, hence $\mathsf{RS}_h \cup \mathsf{RS}_g \subseteq \mathsf{RS}_{h\times g}$; in particular, window information and another finite feature can be combined without making the typing target-specific.

Yoshinaka [16, Proposition 1] showed that fixed-window substitutable languages are not closed under intersection with regular languages or inverse images under homomorphisms that may erase. In contrast, they are closed under inverse images of nonerasing homomorphisms. The following proposition shows that regular filtering and erasing inverse images can be accommodated by changing the finite typing.

Proposition 3.2 (closure under finite information)**.** *Let $h : \Sigma^* \to M$ and $g : \Sigma^* \to N$ be finite-monoid homomorphisms.*

(i) *If $L \in \mathsf{RS}_h$ and $J \in \mathsf{RS}_g$, then $L \cap J \in \mathsf{RS}_{h\times g}$.*

(ii) *If $Q = g^{-1}(F)$ is regular and $L \in \mathcal{C}_h^{\mathrm{cf}}$, then $L \cap Q \in \mathcal{C}_{h\times g}^{\mathrm{cf}}$.*

(iii) *Let $\varphi : \Gamma^* \to \Sigma^*$ be any homomorphism, possibly erasing. Let $B_\varphi = \{1, \zeta\}$ be the two-element monoid with zero ζ, define $e_\varphi(w) = 1$ iff $\varphi(w) = \lambda$ and $e_\varphi(w) = \zeta$ otherwise, and put $\widehat{h} = (h \circ \varphi) \times e_\varphi$. If $L \in \mathsf{RS}_h$, then $\varphi^{-1}(L) \in \mathsf{RS}_{\widehat{h}}$; if moreover $L \in \mathcal{C}_h^{\mathrm{cf}}$, then $\varphi^{-1}(L) \in \mathcal{C}_{\widehat{h}}^{\mathrm{cf}}$.*

Proof. For (i), suppose nonempty x, y have equal $(h\times g)$-type and share a context in $L \cap J$. They then have equal h-type and a common context in L, and equal g-type and a common context in J. Hence $\mathcal{D}_L(x) = \mathcal{D}_L(y)$ and $\mathcal{D}_J(x) = \mathcal{D}_J(y)$, so $\mathcal{D}_{L\cap J}(x) = \mathcal{D}_{L\cap J}(y)$. Item (ii) follows from (i), Proposition 3.1, and closure of CFLs under intersection with regular languages.

For (iii), e_φ is a homomorphism because $\varphi(uv) = \lambda$ iff both $\varphi(u)$ and $\varphi(v)$ are empty. Let equal-$\widehat{h}$-type nonempty x, y share a context in $\varphi^{-1}(L)$. If $e_\varphi(x) = e_\varphi(y) = 1$, then $\varphi(x) = \varphi(y) = \lambda$, so the two factors have identical distributions in the inverse image. Otherwise both images

are nonempty, have equal h-type, and the shared inverse-image context maps to a common context of $\varphi(x), \varphi(y)$ in L. Fixed-h substitutability in L gives equal distributions there, and pulling contexts back through φ gives $\mathcal{D}_{\varphi^{-1}(L)}(x) = \mathcal{D}_{\varphi^{-1}(L)}(y)$. The context-free statement uses closure of CFLs under inverse homomorphism. □

When is a typing useful? For correctness relative to a target L, Definition 2.1 says exactly that h must separate every pair of nonempty factors that share a context but have different distributions. Equivalently, choosing a useful typing is a finite-index monoid-congruence separation problem: no such distributional conflict may be collapsed by $\ker h$. Typing refinement is therefore safe, and product typing combines independent finite features because a conflicting pair need only be separated by one component. Lemma 9.3 later gives the corresponding obstruction: an infinite pairwise family of conflicting factors cannot be separated by any finite typing. Quantitative usefulness is a separate issue. Refinement can enlarge the typed grammar used by the reconstruction bound; Proposition 6.3 exhibits an exponential source-to-typed thickness gap, whereas Sections 7 and 8 give sufficient regimes in which that cost is controlled.

If $\ker h$ restricted to Σ^+ refines the syntactic congruence $\equiv_L$, then $L \in \mathsf{RS}_h$. The converse already fails for Clark–Eyraud substitutability. Consider $L_{\mathrm{c}} := \{a^n c b^n : n \geq 0\}$. Under the nonempty-fragment convention of Definition 2.1, L_{c} is Clark–Eyraud substitutable, but $\equiv_{L_{\mathrm{c}}}$ has infinite index: for $1 \leq i < j$ the context (a, cb^{i+1}) distinguishes a^i from a^j.

3.1 Classical special cases

Clark–Eyraud's original Definition 5 includes all fragments [9]; we follow Clark [21] in excluding λ. This distinction matters even for $L = a^+$: (a, λ) belongs to both $\mathcal{D}_L(\lambda)$ and $\mathcal{D}_L(a)$, whereas (λ, λ) belongs to $\mathcal{D}_L(a)$ but not to $\mathcal{D}_L(\lambda)$. Thus comparing λ with nonempty factors would exclude a^+. Call L *Clark–Eyraud substitutable* when $\mathcal{D}_L(x) \cap \mathcal{D}_L(y) \neq \emptyset$ implies $\mathcal{D}_L(x) = \mathcal{D}_L(y)$ for all $x, y \in \Sigma^+$. This is exactly the trivial-monoid case of Definition 2.1, and hence also Yoshinaka's $(0, 0)$ case.

For $k, \ell \geq 0$, let $\mathrm{pref}_k(w)$ and $\mathrm{suf}_\ell(w)$ denote the prefix and suffix of lengths $\min\{k, |w|\}$ and $\min\{\ell, |w|\}$, respectively; in particular, $\mathrm{pref}_0(w) = \mathrm{suf}_0(w) = \lambda$. Renaming Yoshinaka's variables so that u_i, v_i denote outer contexts, L is (k, ℓ)-*substitutable* if, for all $p \in \Sigma^k$, $q \in \Sigma^\ell$, and $u_1, u_2, v_1, v_2, r, s \in \Sigma^*$ with $prq, psq \neq \lambda$,

$$u_1 prq v_1 \in L, \quad u_1 psq v_1 \in L, \quad u_2 prq v_2 \in L \\ \implies \quad u_2 psq v_2 \in L.$$

Proposition 3.3. *For every fixed $k, \ell \geq 0$ there is a finite monoid homomorphism $h_{k,\ell}$ such that, for every $L \subseteq \Sigma^*$, L is (k, ℓ)-substitutable iff it is $\sim_{h_{k,\ell}}$-substitutable.*

Proof. Put $m_0 := \max\{1, k + \ell\}$, let $h_{k,\ell}(w) := w$ for $|w| < m_0$, and otherwise let $h_{k,\ell}(w) := \langle \mathrm{pref}_k(w), \mathrm{suf}_\ell(w) \rangle$, with a tag separating the two cases. Denote the finite image by $M_{k,\ell} := h_{k,\ell}(\Sigma^*)$. On $M_{k,\ell}$ define, for x, y with $|x|, |y| < m_0$,

$$x \cdot y := h_{k,\ell}(xy), \qquad x \cdot \langle p, q \rangle := \langle \mathrm{pref}_k(xp), q \rangle, \\ \langle p, q \rangle \cdot y := \langle p, \mathrm{suf}_\ell(qy) \rangle, \quad \langle p, q \rangle \cdot \langle p', q' \rangle := \langle p, q' \rangle.$$

Then $h_{k,\ell}(xy) = h_{k,\ell}(x) \cdot h_{k,\ell}(y)$; surjectivity gives associativity and $h_{k,\ell}(\lambda)$ is the identity.

For nonempty x, y, equal types mean either $x = y$ or $x = prq$, $y = psq$ for a common length-k prefix p and length-ℓ suffix q (including $k = \ell = 0$). If L is (k, ℓ)-substitutable, a shared context plus any context of one factor instantiates Yoshinaka's four-word condition, and symmetry gives equal distributions. Conversely, in any Yoshinaka premise the factors $x := prq$ and $y := psq$ are nonempty and lie in the long-word part of the coding, hence have equal $h_{k,\ell}$-type; fixed-$h_{k,\ell}$ substitutability transfers the third context and yields the fourth word. □

Fixed-window counterexample criterion. If x, y have length at least $\max\{1, k+\ell\}$, the same length-k prefix and length-ℓ suffix, a common context, and unequal distributions, then L is not (k,ℓ)-substitutable: the first three conditions give equal $h_{k,\ell}$-types and a shared context, while the last contradicts $\sim_{h_{k,\ell}}$-substitutability.

Thus the fixed (k,ℓ) class is exactly $\mathsf{RS}_{h_{k,\ell}}$. Section 7 gives its polynomial thick-data bound. For $k = \ell = 0$, $h_{0,0}$ merely separates λ from nonempty words and induces the same substitutability condition as the one-element monoid. For an alphabet Γ, set

$$\mathsf{KL}(\Gamma) := \bigcup_{k,\ell \geq 0} \{L \subseteq \Gamma^* : L \text{ is } (k,\ell)\text{-substitutable}\},$$

and abbreviate $\mathsf{KL} := \mathsf{KL}(\Sigma)$.

3.2 Main theorem

Theorem 3.4 (fixed-h learning theorem)**.** *Fix an efficiently computable homomorphism $h : \Sigma^* \to M$ into a finite monoid. There are a set-driven reconstruction operator $\mathcal{B}_h$ and a conservative sequential learner $\mathcal{A}_h$ such that:*

(i) *$\mathcal{B}_h$ is sample-consistent and $\mathcal{B}_h(K)$ is constructible in time polynomial in $\|K\|$;*

(ii) *every $L \in \mathcal{C}_h^{\mathrm{cf}}$ has a finite characteristic sample for $\mathcal{B}_h$;*

(iii) *$\mathcal{A}_h$ identifies every nonempty $L \in \mathcal{C}_h^{\mathrm{cf}}$ in Gold's sense, each update taking time polynomial in the data seen so far;*

(iv) *if the positive image semigroup $h(\Sigma^+)$ divides a product of a finite locally trivial semigroup and a finite group (that is, is a homomorphic image of a subsemigroup of that product), then for every nonempty target the sample in (ii) can be chosen polynomial in $|G_*|$ and τ_{G_*} for any reduced CFG G_* representing that target; this includes every fixed-window and every finite-group typing;*

(v) *for arbitrary fixed h and $L \in \mathcal{C}_h^{\mathrm{lin}}$, the sample in (ii) can be chosen polynomial in $|G_*|$ for any linear CFG G_* representing L.*

The polynomials in (iv)–(v) may depend on the fixed h and Σ, but not on the chosen target grammar G_*. The quantitative mechanism behind items (iv)–(v), together with its limitation for arbitrary fixed typings, is developed in Sections 6–8.

This is consistent with Gold's theorem: on a nonempty alphabet choose distinct $x, y \in \Sigma^+$ with $h(x) = h(y)$, and choose $z \in \Sigma^*$ with $|z| > \max\{|x|, |y|\}$. The finite language $\{x, y, zx\}$ is not $\sim_h$-substitutable: x, y share the empty context but only x admits (z, λ).

Algebraic position of fixed windows. The following positive-image/kernel criterion is classical [6, Propositions XI.4.17 and XIV.1.20]; the generalized definite language characterization is likewise known [4, Proposition 5.18]. Its learning-specific application is Corollary 7.4.

Proposition 3.5 (locally trivial typings and fixed windows)**.** *Let $h : \Sigma^* \to M$ be a finite-monoid homomorphism on a nonempty alphabet, and put $n := |h(\Sigma^+)| + 1$. The following are equivalent:*

(i) *$h(\Sigma^+)$ is locally trivial;*

(ii) *there are $k, \ell \geq 0$ such that $\ker h_{k,\ell}|_{\Sigma^+} \subseteq \ker h|_{\Sigma^+}$.*

If these conditions hold, $(k,\ell) = (n,n)$ works. Moreover every positive image semigroup $h_{k,\ell}(\Sigma^+)$ is locally trivial, and therefore

$$\mathsf{KL}(\Sigma) = \bigcup_{\substack{h:\Sigma^* \to M \\ h(\Sigma^+) \text{ locally trivial}}} \mathsf{RS}_h(\Sigma).$$

The same equality holds after intersecting both sides with CFL.

Proof. Assume (i) and put $S = h(\Sigma^+)$, $n = |S| + 1$. Then $S^n = SE(S)S$, and local triviality also implies $esf = ef$ for all $e, f \in E(S)$ and every $s \in S$ [6, Corollary II.6.35 and Proposition XI.4.17]. Thus for $p, q \in \Sigma^n$ we may write $h(p) = aeb$, $h(q) = cfd$, with e, f idempotent, and obtain $h(prq) = a\,e\,(b\,h(r)\,c)\,f\,d = aefd = h(pq)$ for every $r \in \Sigma^*$. The definition of $h_{n,n}$, which distinguishes short positive words individually, gives (ii).

Conversely, if S is not locally trivial, choose an idempotent $e \in S$ and $m \in S$ with $eme \neq e$, represented by nonempty words r, z, respectively. Given k, ℓ, take $j \geq 1$ with $j|r| \geq \max\{k, \ell\}$. Then $x = r^{2j+1}$ and $y = r^j z r^j$ have the same $h_{k,\ell}$-value, but $h(x) = e \neq eme = h(y)$. Hence (ii) fails.

Finally, the multiplication in Proposition 3.3 shows that every idempotent of the positive image of $h_{k,\ell}$ has the form $\langle p, q\rangle^2$ and satisfies $\langle p, q\rangle t \langle p, q\rangle = \langle p, q\rangle$ for every positive-image element t; thus that semigroup is locally trivial. The forward implication and the kernel-refinement monotonicity give $\mathsf{RS}_h \subseteq \mathsf{RS}_{h_{n,n}}$ whenever $h(\Sigma^+)$ is locally trivial. Taking unions yields the stated equality, also after restriction to CFL. □

4 Reconstruction Operator and Sequential Learner

4.1 Finite-sample reconstruction

For a finite sample $K \subseteq \Sigma^*$, let

$$\widehat{V}(K) := \{[x] : x \in \Sigma^+,\ \exists u, v \in \Sigma^* \text{ with } uxv \in K\}.$$

Thus there is one hypothesis nonterminal for each distinct nonempty factor observed in the sample; the brackets are part of the nonterminal name and do not denote a quotient class. For observed nonempty factors x, y, write

$$x \approx_{K,h} y \quad \Longleftrightarrow \quad h(x) = h(y) \ \text{ and } \ \mathcal{D}_K(x) \cap \mathcal{D}_K(y) \neq \emptyset.$$

The grammar $\mathcal{B}_h(K)$ has nonterminal set $\widehat{V}(K) \cup \{\widehat{S}\}$, start symbol $\widehat{S}$, and the following productions whenever every displayed bracketed nonterminal belongs to $\widehat{V}(K)$:

$$\begin{aligned}
&\text{(B)} && [xy] \to [x][y], && x, y \in \Sigma^+,\\
&\text{(U)} && [x] \to [y], && x \approx_{K,h} y,\\
&\text{(L)} && [a] \to a, && a \in \Sigma,\\
&\text{(S)} && \widehat{S} \to [w], && w \in K \cap \Sigma^+.
\end{aligned}$$

If $\lambda \in K$, Rule (S) also contains $\widehat{S} \to \lambda$.

The construction follows the substring-indexed grammar scheme of Yoshinaka [16, Section 4, Algorithm 1], also recalled by Clark [21, Section 4, Algorithm 1]. For $h = h_{k,\ell}$, the nontrivial instances of Rule (U) coincide with Yoshinaka's unary rules: distinct observed nonempty factors are connected when they share a sample context and have equal $h_{k,\ell}$-type; equivalently, both factors have length at least $\max\{1, k + \ell\}$ and have the same length-k prefix and length-ℓ suffix. The constructions agree up to redundant identity rules, the representation of initial productions by a separate start symbol, and the treatment of λ. For general fixed h, the prefix–suffix test is replaced by equality of h-types.

Lemma 4.1 (sample consistency)**.** *For every finite sample K, $K \subseteq L(\mathcal{B}_h(K))$.*

Proof. If $w = a_1 \cdots a_m \in K \cap \Sigma^+$, Rule (S) enters $[w]$. Repeated use of Rule (B) along any binary factorization of w reaches the one-letter factors, all of which are observed because they occur inside the same sample word. Rule (L) then emits $a_1 \cdots a_m$. If $\lambda \in K$, the start rule emits it directly. □

[2] A short-word element x of the positive image has $0 < |x| < m_0$. For sufficiently large j, $h_{k,\ell}(x^j)$ has long-word type, whereas idempotence of x would force $h_{k,\ell}(x^j) = x$, a contradiction.

Theorem 4.2 (soundness)**.** *If $K \subseteq L$ and L is $\sim_h$-substitutable, then $L(\mathcal{B}_h(K)) \subseteq L$.*

Proof. Induct on the height of a derivation tree rooted at $[x]$ to prove

$$[x] \Rightarrow^*_{\mathcal{B}_h(K)} w \quad \Longrightarrow \quad x \equiv_L w \ \text{ and } \ h(w) = h(x). \tag{1}$$

Every factor nonterminal is indexed by a nonempty observed word, so the fragments compared by Rule (U) satisfy the nonempty requirement of Definition 2.1. Moreover, Rules (B)–(L) cannot derive λ from a factor nonterminal. Rule (L) is immediate. For Rule (U), suppose $[x] \to [y]$. By definition there is a sample context (u, v) with $uxv, uyv \in K \subseteq L$ and $h(x) = h(y)$. Fixed-h substitutability gives $x \equiv_L y$; the induction hypothesis gives $y \equiv_L w$ and $h(w) = h(y)$, proving (1).

For Rule (B), write the derived word as $w_1 w_2$ from $[xy] \to [x][y]$. The induction hypotheses give $x \equiv_L w_1$, $y \equiv_L w_2$, $h(w_1) = h(x)$, and $h(w_2) = h(y)$. Because $\equiv_L$ is a monoid congruence, $xy \equiv_L w_1 w_2$, and the homomorphism law gives $h(w_1 w_2) = h(xy)$.

Finally, if a start rule $\widehat{S} \to [x]$ is used, then $x \in K \subseteq L$, so $(\lambda, \lambda) \in \mathcal{D}_L(x)$. The invariant gives $\mathcal{D}_L(w) = \mathcal{D}_L(x)$ and hence $w \in L$. The direct λ rule is present only when $\lambda \in K \subseteq L$. □

4.2 The sequential learner

Recomputing $\mathcal{B}_h$ after every datum need not stabilize syntactically: for $L_{\mathrm{c}} = \{a^n c b^n : n \geq 0\}$, longer examples introduce new factor nonterminals such as $[a^n c b^n]$. We therefore use the conservative wrapper of Clark–Eyraud's Substitution Graph Learner (SGL, Algorithm 2) and Yoshinaka's (k, ℓ)-Substitutable Grammar Learner ((k, ℓ)-SGL, Algorithm 1) [9, 16]; see also Eyraud–Heinz–Yoshinaka [22, Appendix] for the distinction between set-driven reconstruction and incremental learning. Starting from the fixed empty grammar H_0, after reading w_n put

$$H_n := \begin{cases} H_{n-1}, & w_n \in L(H_{n-1}), \\ \mathcal{B}_h(K_n), & w_n \notin L(H_{n-1}), \end{cases} \qquad \mathcal{A}_h(T[n]) := H_n.$$

Every rebuild uses the whole accumulated sample K_n. Although $\mathcal{B}_h$ is set-driven, the grammar on which $\mathcal{A}_h$ stabilizes may depend on the presentation order; only eventual syntactic stabilization to a grammar for the target is claimed (Corollary 5.5). This order dependence is genuine: for any infinite target and a convergent text T, choose $w \in L$ longer than all examples at the last rebuild. The limiting grammar for T omits $[w]$, but prepending w to T puts $[w]$ in every rebuilt grammar, so the two limiting grammars differ. Clark [21, Section 4, Example 1] illustrates the same presentation-order dependence for $L_{\mathrm{c}} = \{a^n c b^n : n \geq 0\}$: different initial positive examples can lead a conservative learner to stabilize on distinct substring-indexed grammars.

5 Finite Witnesses, Exact Reconstruction, and Complexity

5.1 Target refinement and finite witnesses

For the completeness proof only, we refine a target grammar by yield type and show that the hypothesis can mimic finitely witnessed typed derivations. We use the following normal form as a proof device.

Definition 5.1 (start-separated binary normal form)**.** *A CFG $G = (V, \Sigma, P, S_0)$ is in* start-separated binary normal form *(SSBNF) if S_0 never occurs on a right-hand side, every start rule is $S_0 \to A$ or $S_0 \to \lambda$, and every non-start rule is either $A \to a$ or $A \to BC$.*

The start separation isolates whole-sentence entry from internal nonterminals and allows λ to be handled at the start symbol.

Every nonempty context-free language has an equivalent reduced SSBNF grammar by the standard start-separation, useless-symbol elimination, λ/unit elimination away from the start, and binarization procedures. The refinement below is therefore defined for nonempty context-free languages; the empty target is represented separately by H_0 and $\mathcal{B}_h(\emptyset)$.

Let $G = (V, \Sigma, P, S_0)$ be a reduced SSBNF grammar for a nonempty language L. Form the following yield-typed grammar with start symbol $\widetilde{S}$ and non-start symbols A_μ with $A \in V \setminus \{S_0\}$ and $\mu \in M$. Here and below, juxtaposition of type indices denotes multiplication in M; thus $\mu\nu = \mu \cdot \nu$. The rules are

$$\begin{array}{ll} \widetilde{S} \to A_\mu & (S_0 \to A \in P,\ \mu \in M), \\ \widetilde{S} \to \lambda & (S_0 \to \lambda \in P), \\ A_{h(a)} \to a & (A \to a \in P), \\ A_{\mu\nu} \to B_\mu C_\nu & (A \to BC \in P,\ \mu, \nu \in M). \end{array}$$

The reduced part of this grammar is denoted $\widetilde{G} = (\widetilde{V}, \Sigma, \widetilde{P}, \widetilde{S})$ and called the *typed refinement*. By construction, every symbol of $\widetilde{G}$ is therefore reachable and productive.

Fix an order on Σ and use the induced *shortlex* order: shorter strings come first, with lexicographic order breaking equal-length ties. For every non-start symbol X of $\widetilde{G}$, let $\omega(X)$ be its shortlex-minimal terminal yield, called the *canonical yield* of X. This is well-defined because $\widetilde{G}$ is reduced.

The yield-type split is used in the present completeness proof: the canonical yield of a typed parent and the concatenated canonical yields of its typed children must have equal h-types to license unary Rule (U) (Theorem 5.3).

The role of this split in the present completeness argument can already be seen before typing. Let $h(w) = |w| \bmod 2$ and let G be the SSBNF grammar with start symbol S_0 and rules $S_0 \to A$, $A \to a \mid BC$, $B \to a$, and $C \to a$. The shortest yield of A is a, of odd type, while the concatenation of shortest yields of B and C is aa, of even type. Without separating A by yield type, the corresponding unary-rule step is therefore not licensed. Splitting by yield type makes the required equality an invariant. By contrast, no types are attached to the surrounding left or right contexts, because the reconstruction operator never tests such types.

Proposition 5.2 (typed lifting and yield invariant)**.** *The typed refinement satisfies* $L(\widetilde{G}) = L(G)$*. For every retained non-start symbol* A_μ*,*

$$L_{\widetilde{G}}(A_\mu) = L_G(A) \cap h^{-1}(\{\mu\}).$$

In particular, $A_\mu \Rightarrow_{\widetilde{G}}^* w$ *implies* $h(w) = \mu$*.*

Proof. Erase type annotations to map every typed derivation to a derivation of G. Conversely, annotate each non-start node of a successful derivation tree of G by the h-type of its terminal yield. The terminal and binary rules above are exactly the bottom-up propagation laws for those types, so the annotated tree is a derivation of the untrimmed yield-typed grammar above; all symbols on a successful tree survive trimming.

For a retained non-start A_μ, erasing annotations proves $L_{\widetilde{G}}(A_\mu) \subseteq L_G(A) \cap h^{-1}(\{\mu\})$. Conversely, fix $w \in L_G(A)$ with $h(w) = \mu$. Since A_μ survives trimming, there is a successful typed derivation tree containing an occurrence of A_μ. Annotate a source derivation $A \Rightarrow_G^* w$ bottom-up by the h-types of its subtree yields and graft the resulting typed subtree at that occurrence. Its root has type μ, so the graft is valid; the resulting global typed tree is successful. Consequently all symbols and rules of the grafted subtree survive trimming, and $A_\mu \Rightarrow_{\widetilde{G}}^* w$. The yield-type invariant also follows directly by induction on a typed derivation. □

For every non-start symbol X of $\widetilde{G}$, choose $\chi(X) = (u_X, v_X)$ among the terminal reaching contexts of X so as to minimize $|u_X| + |v_X|$, breaking ties by the shortlex orders of u_X and

then v_X; call $\chi(X)$ the *canonical context* of X. Such a terminal reaching context exists because $\widetilde{G}$ is reduced: reachability places X in a reachable sentential form, and productivity of the surrounding nonterminals lets all symbols to its left and right be expanded to terminal strings. Proposition 5.2 gives $h(\omega(A_\mu)) = \mu$ for every typed non-start symbol.

Define the finite witness set

$$\begin{aligned}\mathsf{W}(\widetilde{G}) := {} & \{u_X\omega(X)v_X : X \in \widetilde{V} \setminus \{\widetilde{S}\}\} \\ & \cup \{u_X a v_X : X \to a \in \widetilde{P}\} \\ & \cup \{u_X\omega(Y)\omega(Z)v_X : X \to YZ \in \widetilde{P}\} \\ & \cup \big(\{\lambda\} \text{ if } \lambda \in L\big).\end{aligned} \tag{2}$$

Every displayed word belongs to L, and the set is finite. In particular, for a binary rule $X \to YZ$ the word in the third family is justified by $\widetilde{S} \Rightarrow^* u_X X v_X \Rightarrow u_X Y Z v_X \Rightarrow^* u_X\omega(Y)\omega(Z)v_X$; the first and second families are justified analogously. We call the elements of $\mathsf{W}(\widetilde{G})$ the *canonical positive witnesses*. The first family records one canonical occurrence of each typed symbol, and the second and third families record the terminal and binary productions, respectively.

5.2 Exact reconstruction and Gold convergence

Theorem 5.3 (completeness from the finite witnesses)**.** *Let G be any reduced SSBNF grammar for a nonempty language L, and let $\widetilde{G}$ be its typed refinement with respect to the fixed homomorphism h. If a finite set K satisfies $\mathsf{W}(\widetilde{G}) \subseteq K$, then $L \subseteq L(\mathcal{B}_h(K))$.*

Neither substitutability nor $K \subseteq L$ is needed for this combinatorial completeness statement.

Proof. For each non-start X write $\widehat{X} := [\omega(X)]$. The corresponding word in the first family of (2) ensures that $\widehat{X}$ exists. We prove by induction on a typed derivation $X \Rightarrow^* w$ that $\widehat{X} \Rightarrow^*_{\mathcal{B}_h(K)} w$.

If the first rule is $X \to a$, the first and second witness families contain $u_X\omega(X)v_X$ and $u_X a v_X$, respectively. Hence $\omega(X) \approx_{K,h} a$ because $h(\omega(X)) = h(a)$ by Proposition 5.2. Rule (U) moves $\widehat{X}$ to $[a]$ when necessary, and Rule (L) yields a.

Suppose the first rule is $X = A_{\mu\nu} \to B_\mu C_\nu$; put $Y := B_\mu$ and $Z := C_\nu$. The first and third witness families contain $u_X\omega(X)v_X$ and $u_X\omega(Y)\omega(Z)v_X$. Since

$$h(\omega(X)) = \mu\nu = h(\omega(Y))h(\omega(Z)) = h(\omega(Y)\omega(Z)),$$

Rule (U) moves $\widehat{X}$ to $[\omega(Y)\omega(Z)]$ when necessary. The third witness also makes $\omega(Y)\omega(Z)$ an observed factor, so Rule (B) gives

$$[\omega(Y)\omega(Z)] \to [\omega(Y)][\omega(Z)].$$

The induction hypotheses finish the two subderivations.

Finally, every nonempty $w \in L$ has a lifted start derivation $\widetilde{S} \to X \Rightarrow^* w$. The start rule itself shows that (λ, λ) is a terminal reaching context of X; its total length is 0, which is minimal, so the canonical choice is $\chi(X) = (\lambda, \lambda)$. The first witness family therefore gives $\omega(X) \in K$, and Rule (S) starts the preceding derivation. The case $w = \lambda$ is immediate from (2) and Rule (S). □

Theorem 5.4 (exact finite-sample reconstruction)**.** *Let $L \in \mathcal{C}_h^{\mathrm{cf}}$ be nonempty, let G be a reduced SSBNF grammar for L, and let $\widetilde{G}$ be its typed refinement. If $\mathsf{W}(\widetilde{G}) \subseteq K \subseteq L$, then $L(\mathcal{B}_h(K)) = L$.*

Proof. Combine Theorems 4.2 and 5.3. □

Thus $\mathsf{W}(\widetilde{G})$ is characteristic for every nonempty target, including $\{\lambda\}$. For $L = \emptyset$, the empty sample gives $L(\mathcal{B}_h(\emptyset)) = \emptyset$. This proves Theorem 3.4(ii).

Corollary 5.5 (Gold identification)**.** *For a fixed homomorphism $h : \Sigma^* \to M$ into a finite monoid M, the learner $\mathcal{A}_h$ identifies every nonempty language in $\mathcal{C}_h^{\mathrm{cf}}$ from positive data. More precisely, for any reduced SSBNF target grammar G and its typed refinement $\widetilde{G}$, once $\mathsf{W}(\widetilde{G}) \subseteq K_{n_0}$, among stages $n \geq n_0$ at most one hypothesis change can occur.*

Proof. Under our text convention the Gold statement concerns nonempty targets; H_0 is the initial empty hypothesis. Let L be nonempty and let T be a text for L. Since $w_1 \notin L(H_0) = \emptyset$, the learner rebuilds at stage 1. Choose n_0 with $\mathsf{W}(\widetilde{G}) \subseteq K_{n_0}$. If the learner rebuilds at some stage $n \geq n_0$, Theorem 5.4 makes the new hypothesis exactly L, after which no positive example can trigger another change. In this case that rebuild is the last hypothesis change. Suppose instead that no rebuild occurs at any stage $n \geq n_0$. Soundness gives $L(H_{n_0}) \subseteq L$. Moreover, $K_n \subseteq L(H_n)$ for every n: a rebuild is sample-consistent, while at a non-rebuild step the newly presented datum is already generated by the unchanged hypothesis. Hence a strict inclusion $L(H_{n_0}) \subsetneq L$ would leave some $w^\star \in L \setminus L(H_{n_0})$ that has not yet occurred by stage n_0. Since every target word eventually occurs in the text, $w^\star$ must occur later and force a rebuild, a contradiction. Thus $L(H_{n_0}) = L$; in this case no hypothesis change after stage n_0 is needed. □

A missed-positive rebuild can be delayed after a characteristic sample appears; compare Eyraud–Heinz–Yoshinaka [22, Appendix].

5.3 Complexity of the reconstruction step

Theorem 5.6 (polynomial-time reconstruction)**.** *For fixed h, $\mathcal{B}_h(K)$ is constructible in time polynomial in $\|K\|$.*

Proof. Put $n_K := \|K\|$ and let $\mathcal{O}_K := \{(u, x, v) \in \Sigma^* \times \Sigma^+ \times \Sigma^* : uxv \in K\}$ be the set of observed nonempty factor occurrences. Since $|\mathcal{O}_K| = O(n_K^2)$, there are at most $O(n_K^2)$ factor nonterminals. Extending factors one symbol at a time from each starting position of each sample word computes and caches all their h-values using $O(n_K^2)$ multiplications in the fixed monoid.

Rule (B) has at most $O(n_K^3)$ candidates, obtained by choosing an observed factor occurrence and a split point. For Rule (U), for each $(u, x, v) \in \mathcal{O}_K$ and each $w \in K$, check whether there is a nonempty word y with $w = uyv$. Such a y, if it exists, is unique; if $h(x) = h(y)$, emit the rule $[x] \to [y]$. This enumerates all Rule (U) candidates, at most $|\mathcal{O}_K||K| = O(n_K^3)$ of them.

Lexical and start rules are lower-order. For each candidate, string matching and explicit encoding and output of the production take $O(n_K)$ time. Duplicate productions need not be removed during enumeration, as they do not affect the generated language. Thus $\mathcal{B}_h(K)$ is constructible in $O(n_K^4)$ time. □

Each update of the conservative learner $\mathcal{A}_h$ is also polynomial in the encoded data seen so far: CFG membership is polynomial in the grammar and input sizes, and both the hypothesis size and any triggered rebuild are polynomial by Theorem 5.6.

Together with Lemma 4.1, Theorem 5.4 and Corollary 5.5, this proves Theorem 3.4(i)–(iii). The quantitative bounds in items (iv)–(v) are proved in Sections 7–8. Grammar size alone cannot bound witness lengths: the standard doubling grammar $S_0 \to D_n$, $D_0 \to a$, $D_i \to D_{i-1}D_{i-1}$ has size $O(n)$ but generates the singleton $\{a^{2^n}\}$, substitutable for every h; compare [9, Example 2] and [22, Example 2.3].

6 Typed Thickness for General Fixed Typings

For an arbitrary fixed h, the finite-witness construction admits a direct quantitative bound once the cost of yield typing is measured explicitly. Let $G = (V, \Sigma, P, S_0)$ be a reduced SSBNF

grammar and let $\widetilde{G}$ be its typed refinement. Put $N := |V \setminus \{S_0\}|$ and $N_t := |\widetilde{V} \setminus \{\widetilde{S}\}|$ and define the *typed thickness* of this presentation by

$$\tau_h^{\mathrm{typ}}(G) := \max\Big(\{\, |\omega(X)| : X \in \widetilde{V} \setminus \{\widetilde{S}\}\,\} \cup \{1\}\Big).$$

Thus $\tau_h^{\mathrm{typ}}(G)$ measures shortest terminal yields after the full yield-type split and trimming. It is a presentation-dependent parameter, not an additional assumption on the learner.

Lemma 6.1 (typed-thickness witness bound)**.** *Let G and $\widetilde{G}$ be as above and write $\tau := \tau_h^{\mathrm{typ}}(G)$. Every non-start typed symbol X has $|u_X|+|v_X| \le (N_t-1)\tau$, and every word $w \in \mathsf{W}(\widetilde{G})$ satisfies $|w| \le (N_t+1)\tau$. If P_{ter} and P_{bin} are respectively the terminal and binary non-start productions of G, then*

$$\|\mathsf{W}(\widetilde{G})\| \le \big(N|M| + |P_{\mathrm{ter}}| + |P_{\mathrm{bin}}||M|^2 + 1\big)\big((N|M|+1)\tau + 1\big).$$

In particular, for fixed h this bound is polynomial in $|G|$ and $\tau_h^{\mathrm{typ}}(G)$.

Proof. Form the directed dependency graph whose vertices are the non-start symbols of $\widetilde{G}$ and whose edges join a parent to each child occurring in a binary production. Since $\widetilde{G}$ is reduced, every non-start symbol X is reachable. Hence some start child A has a path to X in this graph. Choose a shortest such path. It repeats no typed symbol, so it has at most $N_t - 1$ edges.

For each edge of the path, choose a witnessing binary production and expand the child not lying on the path to its canonical yield. Starting from $\widetilde{S} \to A$, these expansions produce a terminal reaching context of X. Each off-path canonical yield has length at most τ, so the total context length is at most $(N_t - 1)\tau$. The canonical context (u_X, v_X) is no longer than this one.

A word in the first family adds one canonical yield to the canonical context; the second family adds one terminal, and the third adds two canonical yields. Since $\tau \ge 1$, every word in (2) therefore has length at most $(N_t + 1)\tau$. Finally, $N_t \le N|M|$; each source terminal production has at most one typed terminal copy, and each source binary production has at most $|M|^2$ typed copies. Thus the displayed first factor bounds the number of witnesses, including the possible witness λ, while the second factor bounds $|w| + 1$ for each witness. Multiplying the two bounds proves the claim. □

Corollary 6.2 (characteristic data from typed thickness)**.** *Fix h. If G is a reduced SSBNF grammar for a nonempty $L(G) \in \mathcal{C}_h^{\mathrm{cf}}$, then $\mathsf{W}(\widetilde{G})$ is a characteristic sample for $\mathcal{B}_h$ whose encoded size is polynomial in $|G|$ and $\tau_h^{\mathrm{typ}}(G)$.*

Proof. Lemma 6.1 gives the quantitative bound, and Theorem 5.4 gives the characteristic-sample property. □

The following example shows a limitation of this typed-refinement bound: typed thickness can grow exponentially relative to source thickness even for a fixed h.

Proposition 6.3 (exponential source-to-typed thickness gap)**.** *There is a fixed two-element typing $h_{\mathtt{c}}$ and a family of reduced SSBNF grammars $G_n^{\mathtt{c}}$ of size $O(n)$ and ordinary thickness 1 such that $\tau_{h_{\mathtt{c}}}^{\mathrm{typ}}(G_n^{\mathtt{c}}) \ge 2^n$. Consequently typed thickness cannot in general be bounded by a polynomial in source grammar size and ordinary thickness, even for one fixed typing.*

Proof. Let $M_{\mathtt{c}} = \{1, \zeta\}$ be the two-element monoid with zero ζ, with $h_{\mathtt{c}}(\mathtt{a}) = 1$ and $h_{\mathtt{c}}(\mathtt{c}) = \zeta$. For $n \ge 1$, define $G_n^{\mathtt{c}}$ by

$$\begin{aligned} &S_0 \to U, \quad U \to \mathtt{a} \mid \mathtt{c} \mid UU \mid E_nD, && D \to \mathtt{c},\\ &E_0 \to \mathtt{a}, \quad E_i \to E_{i-1}E_{i-1} \mid \mathtt{c} \quad (1 \le i \le n). \end{aligned}$$

These are reduced SSBNF grammars of size $O(n)$ with $L(G_n^{\mathtt{c}}) = \{\mathtt{a}, \mathtt{c}\}^+ \in \mathcal{C}_{h_{\mathtt{c}}}^{\mathrm{cf}}$.

Every nonterminal has a terminal yield of length 1: S_0, U, E_0 derive $\mathtt{a}$, and D, E_i $(i \geq 1)$ derive $\mathtt{c}$. Hence the ordinary thickness is $\tau_{G_n^{\mathtt{c}}} = 1$.

In contrast, after refinement to type 1, $E_i \to \mathtt{c}$ is unavailable because $h_{\mathtt{c}}(\mathtt{c}) = \zeta \neq 1$. Since $1 \cdot 1$ is the only product equal to 1, induction gives $L((E_i)_1) = \{\mathtt{a}^{2^i}\}$. Moreover, $\widetilde{S} \to U_\zeta \to (E_n)_1 D_\zeta$ shows that $(E_n)_1$ survives trimming. Thus $\tau_{h_{\mathtt{c}}}^{\mathrm{typ}}(G_n^{\mathtt{c}}) \geq 2^n$.

□

The target language $\{\mathtt{a}, \mathtt{c}\}^+$ in this family does not depend on n. Thus the proposition is not an algorithm-independent lower bound on characteristic data. Its role is only to show that ordinary source thickness does not control the typed-refinement parameter appearing in Corollary 6.2; accordingly, the general fixed-h bound is stated in terms of typed thickness.

7 Polynomial Thick Data Beyond Fixed Windows

Yoshinaka [16, Theorem 1] already established polynomial thick-data learnability for fixed (k, ℓ)-substitutable context-free languages using (k, ℓ)-Greibach normal form and characteristic-set bounds [16, Lemmas 2 and 7]. Here we instead bound the shortest yields of the typed refinement used by $\mathcal{B}_h$. One argument covers both fixed windows and typings that additionally track finite-group values. The corresponding batch bounds are in the polynomial-time-and-thick-data (IPTtD) sense of [16] and [22, Definition 2.12]; we claim no improvement in Yoshinaka's polynomial degree. For general h, ordinary thickness cannot bound typed thickness polynomially (Proposition 6.3).

Fix $k, \ell \geq 0$ and a morphism $g : \Sigma^* \to Q$ into a finite group. Put $m := k + \ell$, $q := |Q|$ and $t := h_{k,\ell} \times g$.

Lemma 7.1 (window–group typed-yield bound)**.** *Let $G = (V, \Sigma, P, S_0)$ be a reduced SSBNF grammar and $N := |V \setminus \{S_0\}| \geq 1$. If a finite typing h satisfies $\ker t|_{\Sigma^+} \subseteq \ker h|_{\Sigma^+}$, then*

$$\tau_h^{\mathrm{typ}}(G) \leq m + \big[1 + 2(m + q - 1)(m + 1)Nq\big]\tau_G.$$

Proof. See Appendix A. □

Theorem 7.2 (polynomial thick data under window–group refinement)**.** *Fix h, Σ and suppose that $\ker(h_{k,\ell} \times g)|_{\Sigma^+} \subseteq \ker h|_{\Sigma^+}$ for some fixed k, ℓ and finite-group morphism g. Every nonempty $L \in \mathcal{C}_h^{\mathrm{cf}}$ represented by a reduced SSBNF grammar G with a non-start symbol has a characteristic sample $\mathsf{W}(\widetilde{G})$ for $\mathcal{B}_h$ whose encoded size is polynomial in $|G|$ and τ_G.*

Proof. Apply Lemma 7.1 and Corollary 6.2. The singleton target $\{\lambda\}$ is handled separately. □

Theorem 7.2 includes fixed windows by taking Q trivial. The following normalization transfers the bound to arbitrary reduced CFG presentations.

Proposition 7.3 (polynomial thickness-preserving SSBNF normalization)**.** *There are fixed polynomials q_{size} and q_{thick} such that every reduced CFG G_* generating a nonempty language can be converted in polynomial time into an equivalent reduced SSBNF grammar G satisfying $|G| \leq q_{\mathrm{size}}(|G_*|)$ and $\tau_G \leq q_{\mathrm{thick}}(|G_*|, \tau_{G_*} + 1)$.*

The proof is in Appendix A.2; terminal isolation and binarization precede non-start λ-elimination so that grammar size and shortest nonempty yields remain polynomially bounded.

Corollary 7.4 (ordinary-thickness bounds for locally trivial–group typings)**.** *Fix $h : \Sigma^* \to M$. Suppose $h(\Sigma^+)$ divides a direct product of a finite locally trivial semigroup and a finite group. Then every nonempty $L \in \mathcal{C}_h^{\mathrm{cf}}$ represented by a reduced CFG G_* has characteristic data for $\mathcal{B}_h$ of encoded size polynomial in $|G_*|$ and τ_{G_*}. This includes all locally trivial positive-image typings and all finite-group typings.*

Proof. Write $h(\Sigma^+)$ as the image of a subsemigroup $T \leq S \times Q$ under a surjective homomorphism $\pi : T \twoheadrightarrow h(\Sigma^+)$, where S is locally trivial and Q is a finite group. Choose $t_a \in \pi^{-1}(h(a))$ for each $a \in \Sigma$, and let $\varphi : \Sigma^+ \to T$ be the semigroup morphism determined by $\varphi(a) = t_a$. Then $\pi \circ \varphi = h|_{\Sigma^+}$. Composing φ with the projections and sending λ to the respective identities defines monoid morphisms $s : \Sigma^* \to S^1$ and $g : \Sigma^* \to Q$, with $h(w) = \pi(s(w), g(w))$ for all $w \in \Sigma^+$. Since $s(\Sigma^+)$ is locally trivial, Proposition 3.5 gives fixed k, ℓ satisfying $\ker h_{k,\ell}|_{\Sigma^+} \subseteq \ker s|_{\Sigma^+}$. Hence $\ker(h_{k,\ell} \times g)|_{\Sigma^+} \subseteq \ker h|_{\Sigma^+}$. Apply Theorem 7.2 and Proposition 7.3. □

The empty target has the empty characteristic sample. Polynomials may depend on the fixed typing and alphabet. We assert neither a converse for the divisor condition, nor bounds uniform in variable windows or groups, nor an incremental polynomial-data guarantee for the sequential learner.

8 The Linear Subclass

Linear languages are exactly the languages accepted by one-turn pushdown automata [19, 23]. The normalization below gives the grammatical counterpart: every successful derivation has one continuing nonterminal path, which we call its *spine*.

Proposition 8.1 (linear-spine SSBNF normalization)**.** *There is a fixed polynomial q_{lin} such that every linear CFG G generating a nonempty language can be converted in polynomial time into an equivalent reduced SSBNF grammar $G_0 = (V_0, \Sigma, P_0, S_0)$ with $|G_0| \leq q_{\mathrm{lin}}(|G|)$ in which every binary production has exactly one fresh terminal-wrapper child W_a with $W_a \to a$ and exactly one other nonterminal child.*

The construction is given in Appendix B. Let $\widetilde{G}_0$ be its typed refinement and put $N_t := |\widetilde{V}_0 \setminus \{\widetilde{S}\}|$. Since each underlying non-start symbol has at most $|M|$ typed copies, $N_t \leq |V_0 \setminus \{S_0\}|\,|M|$.

Lemma 8.2 (short canonical yields)**.** *Every typed non-start symbol X of $\widetilde{G}_0$ satisfies $|\omega(X)| \leq N_t$. Hence $\tau_h^{\mathrm{typ}}(G_0) \leq N_t$.*

The proof is given in Appendix C.

Theorem 8.3 (polynomial characteristic data for linear targets)**.** *For fixed h, every $L \in \mathcal{C}_h^{\mathrm{lin}}$ represented by a linear CFG G has a characteristic sample for $\mathcal{B}_h$ whose $\|\cdot\|$-size is polynomial in $|G|$.*

Proof. For $L = \emptyset$ and $L = \{\lambda\}$ use the samples from Section 5. Otherwise take G_0 from Proposition 8.1. Lemma 8.2 gives $\tau_h^{\mathrm{typ}}(G_0) \leq N_t \leq |V_0 \setminus \{S_0\}|\,|M|$, and Corollary 6.2 yields a characteristic sample polynomial in $|G_0| \leq q_{\mathrm{lin}}(|G|)$. □

Together with Theorem 5.6, this gives polynomial-time batch reconstruction and polynomial characteristic data relative to a chosen linear CFG for targets in $\mathcal{C}_h^{\mathrm{lin}}$. This does not contradict de la Higuera's negative result for the unrestricted class of linear languages [7, Theorem 2]: our theorem concerns only the fixed-h substitutable subclass. Moreover, no incremental polynomial-data guarantee for $\mathcal{A}_h$ is asserted.

A nonregular linear fixed-h example. The preceding theorem is not confined to regular targets, nor to the Clark–Eyraud or fixed-window classes. Define

$$L_{\pm,e} := \{a^{2m} c b^{2m} : m \geq 0\} \cup \{a^{2m+1} d b^{2m+1} : m \geq 0\} \cup \{a^n e b^n : n \geq 0\}.$$

It is generated by the linear grammar

$$S \to U_{\mathrm{ev}} \mid U_e, \quad U_{\mathrm{ev}} \to c \mid aU_{\mathrm{odd}}b, \quad U_{\mathrm{odd}} \to d \mid aU_{\mathrm{ev}}b, \quad U_e \to e \mid aU_eb.$$

The subscripts ev and odd track the parity of enclosing a, b-pairs from the start, not the length parity of the corresponding nonterminal's own yield.

Call c, d, e the *center symbols*.

Proposition 8.4. *The language $L_{\pm,e}$ is linear, nonregular, and belongs to $\mathcal{C}_h^{\mathrm{lin}}$ for some homomorphism h into a finite monoid. It is neither Clark–Eyraud substitutable nor (k, ℓ)-substitutable for any fixed $k, \ell \geq 0$.*

Proof. Linearity follows from the displayed grammar. To establish finite-typed substitutability, put

$$L_{\mathrm{all}} := \{a^n \xi b^n : n \geq 0,\ \xi \in \{c, d, e\}\}, \qquad Q := (aa)^*c(bb)^* \cup a(aa)^*d\,b(bb)^* \cup a^*eb^*.$$

The linear language L_{all} is Clark–Eyraud substitutable under the nonempty-factor convention: a shared context uniquely determines a center-free factor, while the distribution of a factor $a^i\xi b^j$ containing one center depends only on $i - j$. Thus $L_{\mathrm{all}} \in \mathcal{C}_{h_0}^{\mathrm{lin}}$ for the trivial typing h_0. Since Q is regular and $L_{\pm,e} = L_{\mathrm{all}} \cap Q$, Proposition 3.2(ii), together with closure of linear languages under regular intersection, gives $L_{\pm,e} \in \mathcal{C}_{h_0 \times g}^{\mathrm{lin}}$ for a finite-monoid homomorphism g recognizing Q.

Moreover, $L_{\pm,e} \cap a^*eb^* = \{a^neb^n : n \geq 0\}$ proves nonregularity. The factors c, e share the empty context, but $(a, b) \in \mathcal{D}_{L_{\pm,e}}(e) \setminus \mathcal{D}_{L_{\pm,e}}(c)$; hence $L_{\pm,e}$ is not Clark–Eyraud substitutable. Finally, fix $k, \ell \geq 0$ and put $m_0 := \max\{1, k+\ell\}$, $w_e := a^keb^\ell$, and $w_c := a^kcb^\ell$. The factors w_e, w_c have the same length-k prefix and length-ℓ suffix and share the context $(a^{2m_0-k}, b^{2m_0-\ell})$, whereas $(a^{2m_0+1-k}, b^{2m_0+1-\ell}) \in \mathcal{D}_{L_{\pm,e}}(w_e) \setminus \mathcal{D}_{L_{\pm,e}}(w_c)$. The fixed-window counterexample criterion gives $L_{\pm,e} \notin \mathsf{KL}$. □

A concrete four-element typing. The regular-filter argument establishes the required existence of a finite typing. In fact, a four-element typing already suffices. Let $M_{\pm,e} := \{1, \kappa_{cd}, \kappa_e, 0\}$ be the monoid with identity 1 and $\mu\nu = 0$ for all $\mu, \nu \in \{\kappa_{cd}, \kappa_e, 0\}$, and define $h_{\pm,e} : \{a, b, c, d, e\}^* \to M_{\pm,e}$ by

$$h_{\pm,e}(a) = h_{\pm,e}(b) = 1, \qquad h_{\pm,e}(c) = h_{\pm,e}(d) = \kappa_{cd}, \qquad h_{\pm,e}(e) = \kappa_e. \tag{3}$$

Then $L_{\pm,e}$ is $\sim_{h_{\pm,e}}$-substitutable. Indeed, two center-free nonempty factors sharing a context must coincide. The distribution of a^ieb^j depends only on $i - j$, while that of $a^i\xi b^j$ for $\xi \in \{c, d\}$ depends only on $(i - j, \mathrm{par}(\xi) - i \bmod 2)$, where $\mathrm{par}(c) = 0$ and $\mathrm{par}(d) = 1$. A shared context forces these invariants to agree, and equal $h_{\pm,e}$-types distinguish e-factors from c/d-factors; thus equal-type factors with a common context have the same distribution.

Thus Theorem 8.3 applies to a nonregular linear language outside KL. The displayed grammar is even-linear, so no separation from Takada's control-set framework is claimed.

Finite-state control induced by yield typing. The following straightforward product construction gives a limited connection with Takada's control-set viewpoint. Give each production of a linear grammar $G = (V, \Sigma, P, S)$ a distinct label from Π, and write $A \Rightarrow_G^{\alpha} w$ when $\alpha \in \Pi^*$ is the production-label word of a derivation. For $F \subseteq M$, let

$$C_{h,F}(G) := \{\alpha \in \Pi^* : S \Rightarrow_G^{\alpha} w \text{ for some } w \in \Sigma^*,\ h(w) \in F\}.$$

This is a regular control set. A finite automaton uses states $V \times M \times M$ (and one accepting state), starts in $(S, 1, 1)$, and for a labeled rule $A \to uBv$ makes the transition $(A, \mu_L, \mu_R) \mapsto$

$(B, \mu_L h(u), h(v)\mu_R)$. A terminal rule $A \to w$ is accepted exactly when $\mu_L h(w)\mu_R \in F$. The invariant is that a sentential form uAv is represented by $(A, h(u), h(v))$, so the accepted label words are exactly those of successful derivations whose yields have h-type in F. Thus regular control by $C_{h,F}(G)$ generates precisely $L(G) \cap h^{-1}(F)$. Starting from any nonterminal A with $F = \{\mu\}$ gives the same interpretation for a typed copy A_μ. This is a finite-state bridge to Takada's regular control languages [18]; it does not claim that arbitrary Takada control sets arise from terminal yield typings.

9 Expressiveness and Boundaries

We now locate the framework between the fixed-window hierarchy and broader context-free families.

9.1 A nonlinear target inside RS

Let $\Delta := \{a^n b^n : n \geq 0\}$ and consider Δ^*. For $z \in \{a, b\}^*$ put $\beta(z) := |z|_a - |z|_b$; the *height* after reading a prefix p is $\beta(p)$. For any word x, define its *admissible entry-height set* by

$$\mathcal{Q}(x) := \{q \in \mathbb{Z}_{\geq 0} : q + \beta(x) \geq 0, \quad a^q x b^{q+\beta(x)} \in \Delta^*\}.$$

This is exactly the set of heights $\beta(u)$ at which x occurs in a word $uxv \in \Delta^*$.

For example, $\mathcal{Q}(ab) = \mathbb{Z}_{\geq 0}$, $\mathcal{Q}(aa) = \mathbb{Z}_{\geq 0}$, $\mathcal{Q}(\lambda) = \mathbb{Z}_{\geq 0}$, $\mathcal{Q}(bb) = \mathbb{Z}_{\geq 2}$, $\mathcal{Q}(ba) = \{1\}$, $\mathcal{Q}(bbaa) = \{2\}$, and $\mathcal{Q}(baaba) = \varnothing$.

The canonical completion gives the membership criterion

$$w \in \Delta^* \iff \beta(w) = 0 \text{ and } 0 \in \mathcal{Q}(w). \tag{$\star$}$$

Let $\mathrm{fst}(w), \mathrm{lst}(w) \in \{\bot, a, b\}$ be the first and last symbols, with $\bot$ on λ, and let $\mathrm{ba}(w)$ record whether w contains ba. Put

$$h_\star(w) := (\mathrm{fst}(w), \mathrm{lst}(w), \mathrm{ba}(w)).$$

The summary of a concatenation is determined by the summaries of its factors: the endpoints concatenate in the evident way and a new ba can arise only across a boundary $b|a$. Hence the finite image $M_\star := h_\star(\{a, b\}^*)$ carries the product $h_\star(x)h_\star(y) := h_\star(xy)$, making $h_\star$ a finite-monoid homomorphism.

Appending a word on the right cannot enlarge the admissible entry-height set; more precisely, the following concatenation identity holds:[3]

$$\mathcal{Q}(st) = \mathcal{Q}(s) \cap \{q \in \mathbb{Z} : q + \beta(s) \in \mathcal{Q}(t)\} \cap \begin{cases} \{-\beta(s)\}, & (\mathrm{lst}(s), \mathrm{fst}(t)) = (b, a), \\ \mathbb{Z}, & \text{otherwise.} \end{cases} \tag{$\star\star$}$$

Proposition 9.1 (a nonlinear fixed-$h_\star$ target)**.** *The language Δ^* is deterministic context-free, nonregular, and nonlinear, and it is $\sim_{h_\star}$-substitutable. Moreover it is not (k, ℓ)-substitutable for any fixed $k, \ell \geq 0$. Hence $\Delta^* \in \mathsf{RS}_{h_\star}(\{a, b\}) \cap \mathrm{CFL}$ and $\Delta^* \in \mathsf{RS}(\{a, b\}) \setminus \mathsf{KL}(\{a, b\})$.*

Proof. The grammar $S \to TS \mid \lambda$, $T \to aTb \mid \lambda$ generates Δ^*. It is deterministic context-free: a DPDA pushes while reading the a-part of a factor $a^n b^n$, pops during its b-part, and permits a new such factor only after the stack returns to its bottom marker. Nonregularity

[3] Indeed, the conditions for reading st from height q split into those within s, namely $q \in \mathcal{Q}(s)$; those within t, namely $q + \beta(s) \in \mathcal{Q}(t)$; and the condition at the boundary $s|t$. Only when this boundary is $b|a$ must the height immediately after that b be 0, requiring $q + \beta(s) = 0$. These conditions are also sufficient, and the identity follows.

follows from $\Delta^* \cap a^*b^* = \Delta$, and nonlinearity from closure of linear languages under regular intersection together with $\Delta^* \cap a^*b^*a^*b^* = \Delta\Delta$. The nonlinearity of $\Delta\Delta$ is explicitly recorded by Autebert–Berstel–Boasson [23, Section 6.1, before Theorem 6.1].

For substitutability, let nonempty x, y have equal $h_\star$-type and a common context (u, v). By the preceding characterization of $\mathcal{Q}$ and additivity of β, the memberships $uxv, uyv \in \Delta^*$ imply $\beta(x) = \beta(y)$ and $\beta(u) \in \mathcal{Q}(x) \cap \mathcal{Q}(y)$. Since $h_\star(x) = h_\star(y)$, either both contain ba or neither does.

In the former case, write $x = rbat$ with $r, t \in \{a, b\}^*$. In any word of Δ^*, the height immediately after the b in each occurrence of ba is 0. Hence every $q \in \mathcal{Q}(x)$ satisfies $q + \beta(rb) = 0$, so $\mathcal{Q}(x)$ has at most one element; the same holds for $\mathcal{Q}(y)$. Since $\beta(u) \in \mathcal{Q}(x) \cap \mathcal{Q}(y)$, we obtain $\mathcal{Q}(x) = \{\beta(u)\} = \mathcal{Q}(y)$.

In the latter case, $x, y \in a^*b^*$, and for each $z \in \{x, y\}$ we have $\mathcal{Q}(z) = \{q \in \mathbb{Z} : q \geq -\min\{0, \beta(z)\}\}$. Since we have already shown $\beta(x) = \beta(y)$, it follows immediately that $\mathcal{Q}(x) = \mathcal{Q}(y)$.

Thus $\beta(x) = \beta(y)$ and $\mathcal{Q}(x) = \mathcal{Q}(y)$. Also, $\mathrm{fst}(x) = \mathrm{fst}(y)$ and $\mathrm{lst}(x) = \mathrm{lst}(y)$ by equality of $h_\star$-types. Applying $(\star\star)$ twice gives, for all u', v', $\beta(u'xv') = \beta(u'yv')$ and $\mathcal{Q}(u'xv') = \mathcal{Q}(u'yv')$. By $(\star)$, $u'xv' \in \Delta^*$ iff $u'yv' \in \Delta^*$. Thus $\mathcal{D}_{\Delta^*}(x) = \mathcal{D}_{\Delta^*}(y)$.

For the fixed-window separation, fix k, ℓ and let $m := \max\{1, k + \ell\}$, $x := a^m b^m$, and $y := a^m b^m a^m b^m$. They have the same length-k prefix and length-ℓ suffix and share the empty context. Moreover $axb \in \Delta^*$, whereas the central ba in ayb has height 1, so $ayb \notin \Delta^*$. The fixed-window counterexample criterion therefore excludes every (k, ℓ). □

9.2 A capped regular counter family

Yoshinaka's regular language $L_0 = ae^*ce^*a \cup ae^*de^*a \cup be^*ce^*b$ already gives a fixed-window separation [16, Proposition 1]; capped counters contrast with the uncapped obstruction below.

Let $\Sigma_c := \{\uparrow, \downarrow, \#\}$. For $w = \delta_1 \cdots \delta_n \in \Sigma_c^*$, let $\mathrm{ht}_i(w)$ be the height after i letters, with $\mathrm{ht}_0(w) = 0$, changing by $+1$ on $\uparrow$, -1 on $\downarrow$, and resetting to 0 on $\#$. For $\rho \geq 1$, put

$$\mathrm{CTR}_\rho := \{w \in \Sigma_c^* : 0 \leq \mathrm{ht}_i(w) \leq \rho \text{ for all } 0 \leq i \leq |w|\}.$$

For every $\rho \geq 1$, CTR_ρ is regular: a deterministic finite automaton records heights $0, \ldots, \rho$, uses one sink outside the range, and resets to 0 on $\#$. Hence $\mathrm{CTR}_\rho \in \mathsf{RS}(\Sigma_c)$ by Proposition 3.1.

Theorem 9.2. *For every $\rho \geq 2$, CTR_ρ is not (k, ℓ)-substitutable for any $k, \ell \geq 0$.*

Proof. Fix k, ℓ and put $x := (\uparrow\downarrow)^k \uparrow^{\rho-1} \#^\ell$ and $y := (\uparrow\downarrow)^k \uparrow^\rho \#^\ell$. They are long enough for the fixed-window criterion, have the same length-k prefix and length-ℓ suffix, and both lie in CTR_ρ. Yet $\uparrow x \in \mathrm{CTR}_\rho$ while $\uparrow y \notin \mathrm{CTR}_\rho$, the latter reaching height $\rho + 1$. Hence their distributions differ, and the criterion applies. □

The bound $\rho \geq 2$ is sharp[4]. The DFA transition monoid gives a typing h_ρ recognizing each regular CTR_ρ; only $\rho \geq 2$ lies outside the fixed-window hierarchy.

9.3 Natural deterministic context-free languages outside RS

Lemma 9.3 (finite-monoid obstruction)**.** *Let $L \subseteq \Sigma^*$. Suppose an infinite $\Xi \subseteq \Sigma^+$ satisfies, for all distinct $x, y \in \Xi$, $\mathcal{D}_L(x) \cap \mathcal{D}_L(y) \neq \emptyset$ and $\mathcal{D}_L(x) \neq \mathcal{D}_L(y)$. Then $L \notin \mathsf{RS}(\Sigma)$.*

Proof. Any finite h identifies two elements of Ξ; their common context and unequal distributions contradict $\sim_h$-substitutability. □

[4]Indeed, CTR_1 is $(1, 1)$-substitutable. Membership is characterized by the initial symbol not being $\downarrow$ and the absence of the length-2 factors $\uparrow\uparrow$, $\downarrow\downarrow$, and $\# \downarrow$. Thus, for nonempty admissible factors with a common context, equality of their first and last symbols ensures equal membership under every context.

Removing the upper cap gives the deterministic context-free language

$$\mathrm{CTR} := \{w \in \Sigma_c^* : \mathrm{ht}_i(w) \geq 0 \text{ for all } 0 \leq i \leq |w|\},$$

accepted by the evident deterministic pushdown counter with reset.

Corollary 9.4 (uncapped counter obstruction)**.** $\mathrm{CTR} \notin \mathsf{RS}(\Sigma_c)$.

Proof. Use $\Xi = \{\uparrow^i : i \geq 1\}$. For $i < j$, $(\lambda, \downarrow^i)$ is common, while $(\lambda, \downarrow^j)$ is a context only of $\uparrow^j$. □

For the one-bracket Dyck language

$$D_1 := \{w \in \{a,b\}^* : |w'|_a \geq |w'|_b \text{ for every prefix } w',\ |w|_a = |w|_b\},$$

a standard deterministic context-free language [24], our notation denotes the whole Dyck language (called D_1^* in the convention of Autebert–Berstel–Boasson [23]).

Corollary 9.5. $D_1 \notin \mathsf{RS}(\{a,b\})$.

Proof. Use $\Xi = \{b^i a^i : i \geq 1\}$. For $i < j$, (a^j, b^j) is common, whereas (a^i, b^i) is a context only of $b^i a^i$; for $b^j a^j$ the prefix $a^i b^j$ underflows. □

Thus unbounded stack behavior alone is not the obstruction: the nonregular one-turn language $L_{\pm,e}$ lies in RS, but an infinite family of mutually comparable factors with distinct contextual behavior cannot be separated by any finite h.

Lemma 9.6 (fixed-word quotient)**.** *If L is $\sim_h$-substitutable and $z \in \Sigma^*$, then $L/z := \{w : wz \in L\}$ is $\sim_h$-substitutable.*

Proof. A common context (u, v) in L/z becomes (u, vz) in L; equality of distributions in L then transfers every context (s, t) by appending z. □

The language generated by $S \to aSS \mid b$ is the *Łukasiewicz language*. Yoshinaka [16, Section 3] already observed that this language is not (k,ℓ)-substitutable for any fixed k, ℓ. In the notation used here, $L_{\mathrm{Luk}} = D_1 b$ [23], and it is deterministic context-free [24]. The quotient argument gives the stronger conclusion that it lies outside the present finite-typing union: if $L_{\mathrm{Luk}} \in \mathsf{RS}(\{a,b\})$, then Lemma 9.6 would put $D_1 = L_{\mathrm{Luk}}/b$ in RS, contradicting Corollary 9.5.

9.4 Comparison with Clark's congruential family

Clark [25] allows a finite initial set $I \subseteq V$; write $L_G[I] := \bigcup_{A \in I} L_G(A)$. Such a grammar is *congruential* if each $L_G(A)$ lies in one syntactic-congruence class of $L_G[I]$. Let $\mathsf{CONG}(\Gamma)$ be the languages over Γ generated by such grammars. Clark already places substitutable and (k,ℓ)-substitutable languages in this family; the following extends the containment to every fixed-h slice.

Proposition 9.7 (comparison with Clark's congruential family)**.** *For every finite alphabet Γ and finite-monoid homomorphism $h : \Gamma^* \to M$, $\mathcal{C}_h^{\mathrm{cf}} \subseteq \mathsf{CONG}(\Gamma)$. Consequently $\mathsf{RS}(\Gamma) \cap \mathrm{CFL} \subseteq \mathsf{CONG}(\Gamma)$, and for $\Gamma = \{a,b\}$ the inclusion is proper.*

Proof. Let nonempty $L \in \mathcal{C}_h^{\mathrm{cf}}$ and take its reduced SSBNF grammar and typed refinement $\widetilde{G}$. Every non-start typed symbol $X = A_\mu$ has a terminal reaching context (u, v), and every $w \in L_{\widetilde{G}}(X)$ is nonempty with $h(w) = \mu$ by Proposition 5.2. Hence any $x, y \in L_{\widetilde{G}}(X)$ have the same type and the common context (u, v); fixed-h substitutability gives $x \equiv_L y$. Thus each typed nonterminal generates strings from one syntactic-congruence class.

Delete the separated start symbol and use its typed successors as the finite initial set; if $\lambda \in L$, add one fresh initial symbol generating only λ. Proposition 5.2 then gives a congruential grammar for L. The empty language is handled by an initial symbol with no productions. This proves the inclusion after taking the union over h.

For properness over $\{a, b\}$, use the Dyck example also discussed by Clark [25]. Every $x \in D_1$ satisfies $uxv \in D_1$ iff $uv \in D_1$, so $x \equiv_{D_1} \lambda$. Hence $S \to aSbS \mid \lambda$ is congruential, while Corollary 9.5 gives $D_1 \notin \mathsf{RS}(\{a, b\})$. □

10 Conclusion

For fixed h, we separate polynomial-time set-driven reconstruction from conservative Gold identification. Canonical witnesses yield characteristic data bounded by typed thickness; ordinary thickness suffices when the positive image divides a product of a finite locally trivial semigroup and a finite group, while linear targets admit a grammar-size bound. The typed-thickness gap limits this refinement analysis, not learning algorithms in general.

After finite refinement of the typing, product typing accommodates regular filtering and arbitrary inverse homomorphisms; this is a compositionality statement rather than closure of one fixed-h slice. The locally trivial criterion recovers the fixed-window union. Nonregular linear and nonlinear examples reach beyond fixed windows; counter/Dyck obstructions and strict containment in Clark's congruential family delimit the framework. Open questions include characterizing $\mathsf{RS} \cap \mathrm{CFL}$, the extent of $\mathcal{C}_h^{\mathrm{lin}}$ for nonregular targets, how to choose useful typings, and direct pushdown characterizations.

Data availability

No empirical data were used for the research described in this article.

Declaration of competing interest

The author declares no known competing financial interests or personal relationships that could have influenced the work reported in this paper.

Funding

This research received no specific grant from any funding agency in the public, commercial, or not-for-profit sectors.

Acknowledgments

The author thanks Makoto Kanazawa and Ryo Yoshinaka for guidance, encouragement, and help with relevant literature, and the two anonymous reviewers for careful and constructive reports that substantially improved the paper, in particular by identifying problems in the submitted learning algorithm and by prompting the direct two-word witness in Theorem 9.2. Any remaining errors are the author's responsibility.

A Fixed-window witness bounds and thickness-preserving normalization

A.1 Proof of Lemma 7.1

Proof of Lemma 7.1. Put $m = k + \ell$, $q = |Q|$ and $N = |V \setminus \{S_0\}|$. It suffices first to bound a shortest word w in each nonempty $L_G(A) \cap t^{-1}(\{\eta\})$, for non-start A. The nodes of an A-derivation tree are nonterminal occurrences; a lexical rule $B \to a$ gives a leaf labelled B. Every node therefore has a non-start label and a nonempty yield. If $m > 0$ and η has a short-word window type, then $|w| < m$. Otherwise $|w| \geq m$, including when $m = 0$. Fix a binary derivation tree T for w. Protect the first k and last ℓ positions, allowing either side to be empty, and denote their union by P, $|P| = m$. For each node z write $I(z)$ for its yield interval, $y(z)$ for its yield, $B(z)$ for its nonterminal label and $M(z) := I(z) \cap P$. Call z *essential* if $M(z) \neq \varnothing$ or $|y(z)| > \tau_G$, and let E be the set of essential nodes. It is ancestor-closed, since marks and yield lengths cannot decrease when moving to a parent.

Height. On any ancestor chain $M(z)$ decreases by inclusion and takes at most $m+1$ values. If two comparable distinct essential nodes have the same triple $(M(z), B(z), g(y(z)))$, replace the upper subtree by the lower one. Proper descent strictly decreases yield length; equal marks preserve the protected positions and equal group values preserve $g(w)$. This contradicts shortestness in the t-fiber. Every essential-node chain thus has at most

$$H := (m+1)Nq$$

nodes.

Width. Let F be an antichain in E. At most m of its nodes meet P. If q further nodes existed, order them left to right as $z_1, \ldots, z_q$. Their disjoint unprotected yields $a_i := y(z_i)$ all have length greater than τ_G. Choose $b_i \in L_G(B(z_i))$ with $|b_i| \leq \tau_G$. If $g(a_i) = g(b_i)$ for any i, replacing that subtree alone contradicts minimality. Thus

$$d_i := g(p_i)\, g(b_i) g(a_i)^{-1} g(p_i)^{-1} \neq 1_Q,$$

where p_i is the prefix of the *original* w preceding a_i. For any index set $J = \{i_1 < \cdots < i_r\}$, simultaneous replacement multiplies $g(w)$ on the left by the ordered product $\prod_{i\in J}^{\rightarrow} d_i := d_{i_1} \cdots d_{i_r}$: expand from right to left so that each original prefix p_i is unaffected when its factor is replaced. This does not require commutativity. Among the $q+1$ prefix products $1_Q, d_1, d_1 d_2, \ldots, d_1 \cdots d_q$, two coincide. Their quotient gives a nonempty consecutive index interval J with $\prod_{i\in J}^{\rightarrow} d_i = 1_Q$. Replacing precisely those subtrees preserves the window and group values while shortening the word, a contradiction. Hence every antichain has at most

$$W := m + q - 1$$

nodes.

Count. The set E occupies at most H depth levels, each an antichain, so $|E| \leq WH$. If E is nonempty, positions outside P are partitioned among maximal nonessential subtrees, each with yield length at most τ_G. There are at most $2|E|$ such subtrees, since each is rooted at a child of an essential node. Every unprotected position belongs to a nonessential lexical leaf (of yield length $1 \leq \tau_G$); protected positions are counted separately by the m term. If E is empty, the root yield has length at most τ_G. In both cases,

$$|w| \leq m + (1 + 2|E|)\tau_G \leq m + \big[1 + 2(m+q-1)(m+1)Nq\big]\tau_G.$$

Taking the maximum bounds the retained t-typed symbols. For a retained h-typed symbol A_μ, choose $w_0 \in L_G(A) \cap h^{-1}(\{\mu\})$, which exists by Proposition 5.2, and put $\eta := t(w_0)$. The kernel inclusion gives

$$L_G(A) \cap t^{-1}(\{\eta\}) \subseteq L_G(A) \cap h^{-1}(\{\mu\}),$$

so the shortest yield in the latter set is no longer than that in the former. This proves the lemma. □

A.2 Thickness-preserving SSBNF normalization

Proof of Proposition 7.3. Let $G_* = (V, \Sigma, P, S)$ be reduced and generate a nonempty language. Put $n := |G_*|$ and $\bar{\tau} := \tau_{G_*} + 1$. We describe a standard normalization in an order that keeps both grammar size and shortest terminal yields polynomial.

First introduce a fresh start symbol S_0 with $S_0 \to S$ and, if $\lambda \in L(G_*)$, retain the possibility $S_0 \to \lambda$ at the end. Isolate every terminal occurring in a right-hand side of length at least two by a fresh wrapper $W_a \to a$, and binarize every right-hand side of length greater than two. Call the resulting grammar G_{bin}. These operations have total size $O(n)$ up to the chosen reasonable encoding. Every original nonterminal keeps the same terminal language. Every fresh binarization symbol expands a contiguous block of some original right-hand side, containing at most $O(n)$ original symbols; replacing each productive nonterminal in that block by a shortest terminal yield gives $\tau_{G_{\mathrm{bin}}} \le c_1 n \bar{\tau}$ for a fixed constant c_1.

We next bound the shortest *nonempty* yield of a nullable symbol of G_{bin}. Suppose $A \Rightarrow^*_{G_{\mathrm{bin}}} w$ for some nonempty terminal word w. Choose one terminal leaf in a derivation tree and follow the path from the root to that leaf. If the same nonterminal occurs twice on this path, replace the subtree at the upper occurrence by the subtree at the lower occurrence. Repeating this shortcut yields a path with at most $|V_{G_{\mathrm{bin}}}|$ nonterminal occurrences. At every binary node on the path, replace the off-path sibling by a shortest terminal yield; a nullable sibling may contribute the empty word. The chosen terminal leaf contributes one symbol. Hence

$$\min\{\, |w| > 0 : A \Rightarrow^*_{G_{\mathrm{bin}}} w \,\} \le 1 + |V_{G_{\mathrm{bin}}}| \tau_{G_{\mathrm{bin}}} \le c_2 n^2 \bar{\tau}$$

for another fixed constant c_2.

Now eliminate all non-start λ-productions. Because G_{bin} is binary, a rule $A \to BC$ creates at most the variants $A \to BC$, $A \to B$, and $A \to C$ according to which children are nullable; there is therefore no exponential nullable-subset expansion. The usual construction preserves, for every nonterminal, its nonempty terminal language, so the preceding bound is unchanged. Next compute unit closure and eliminate non-start unit productions. For each source nonterminal and each unit-reachable non-unit production, copy that production to the source. With $O(n)$ nonterminals and $O(n)$ non-unit productions before this step, this creates at most $O(n^2)$ rules. Unit elimination preserves every nonterminal language and therefore does not increase the thickness bound. Finally trim useless symbols.

The resulting grammar is reduced and has separated start form; every non-start rule is terminal or binary, and $S_0 \to \lambda$ is the only possible λ-rule. Thus it is in SSBNF. Its size is polynomial in n, and its thickness is at most $c_2 n^2(\tau_{G_*} + 1)$. All transformations above are computable in polynomial time, proving the proposition. □

B Proof of the linear-spine SSBNF normalization

The construction for Proposition 8.1 combines standard start-symbol separation, elimination of useless, nullable, and unit rules, and binarization, with the cases written out to preserve the single-spine form needed in Section 8.

Proof of Proposition 8.1. If $L(G) = \{\lambda\}$, take the reduced grammar with the single rule $S_0 \to \lambda$; it has no binary productions, so the required condition is vacuous. Hence assume below that $L(G)$ contains a nonempty word.

Let S be the original start symbol. Introduce a fresh S_0 that never occurs on a right-hand side, with $S_0 \to S$ and, if $\lambda \in L(G)$, $S_0 \to \lambda$. Treat S thereafter as a non-start symbol.

Remove useless symbols, eliminate non-start λ-rules while preserving λ only at S_0, and then eliminate non-start unit rules. Unit elimination is not applied to the start rules, so every start rule surviving final trimming has the required form $S_0 \to A$ or $S_0 \to \lambda$. This order also removes unit rules created by λ-elimination. Linearity implies that nullable elimination creates at most one additional alternative per production and leaves at most one nonterminal on every surviving right-hand side. Unit closure copies at most one non-unit production for each source/reachable-production pair, so these preprocessing steps remain polynomial.

Binarize using fresh terminal wrappers $W_a \to a$, without introducing unit rules. For each rule $A \to a_1 \cdots a_m B b_1 \cdots b_n$, unit elimination ensures $m + n > 0$. Put $R_0 := B$ and, for $n > 0$, form a right chain

$$R_j \to R_{j-1} W_{b_j} \qquad (1 \le j \le n).$$

All intermediate R_j are fresh. If $m = 0$ let $R_n := A$, so the last displayed rule has left side A; otherwise R_n is fresh for $n > 0$, and $R_0 = B$ when $n = 0$. For $m > 0$ put $L_m := R_n$, introduce fresh $L_1, \ldots, L_{m-1}$, and use

$$A \to W_{a_1} L_1, \qquad L_i \to W_{a_{i+1}} L_{i+1} \quad (1 \le i < m).$$

Here $L_1 = R_n$ when $m = 1$. Each rule is binary with precisely one fresh terminal-wrapper child; the original symbol B is not a wrapper.

For a terminal-only rule $A \to a_1 \cdots a_m$, keep $A \to a_1$ when $m = 1$. For $m \ge 2$, introduce fresh $\Theta_2, \ldots, \Theta_m$ and use

$$A \to W_{a_1} \Theta_2, \qquad \Theta_i \to W_{a_i} \Theta_{i+1} \quad (2 \le i \le m-1), \qquad \Theta_m \to a_m.$$

(When $m = 2$, the middle family is empty.) Every new non-start rule is therefore either terminal or binary with exactly one wrapper child and one other nonterminal child. By construction, a wrapper W_a occurs only in the wrapper-child position of a binary rule and never as its continuing nonterminal child; the terminal-only construction ends instead in a fresh non-wrapper nonterminal with a terminal rule. Final trimming preserves this property, the separated start form, and reducedness. The wrapper and chain factorizations are linear in the already polynomial total right-hand-side length, so the whole construction has polynomial size and runs in polynomial time. □

C Proof of the short canonical-yield lemma

Proof of Lemma 8.2. In the linear-spine SSBNF form of Proposition 8.1, each binary spine step has the form $A \to W_a B$ or $A \to B W_a$, with $W_a \Rightarrow a$, and therefore emits exactly one terminal outside the continuing nonterminal.

If X is a terminal-wrapper symbol then $|\omega(X)| = 1$. Otherwise choose a shortest terminal derivation from the typed spine symbol X and write its spine as $X = X_0, X_1, \ldots, X_t$, where X_t uses a terminal rule. Its yield has length $t + 1$. No typed spine symbol can repeat: if $X_i = X_j$ with $i < j$, deleting the segment between the two occurrences gives a valid derivation with fewer spine steps and hence a shorter yield. Thus the $t + 1$ typed spine symbols are distinct non-start symbols, so $t + 1 \le N_t$. Hence $|\omega(X)| \le N_t$ in all cases. □

References

[1] M. Kanazawa. *Learnable Classes of Categorial Grammars.* CSLI Publications, 1998.

[2] M. Kanazawa. Identification in the limit of categorial grammars. *Journal of Logic, Language and Information*, 5(2):115–155, 1996. doi:10.1007/BF00173697.

[3] E. M. Gold. Language identification in the limit. *Information and Control*, 10(5):447–474, 1967.

[4] J.-E. Pin. Syntactic semigroups. In G. Rozenberg and A. Salomaa (eds.), *Handbook of Formal Languages*, Vol. 1, pp. 679–746, Springer, 1997.

[5] S. Eilenberg. *Automata, Languages, and Machines*, Vol. A. Academic Press, 1974.

[6] J.-E. Pin. *Mathematical Foundations of Automata Theory*. Lecture notes, version of March 24, 2025. `https://www.irif.fr/~jep/PDF/MPRI/MPRI.pdf`.

[7] C. de la Higuera. Characteristic sets for polynomial grammatical inference. *Machine Learning*, 27:125–138, 1997.

[8] C. de la Higuera. *Grammatical Inference: Learning Automata and Grammars*. Cambridge University Press, 2010.

[9] A. Clark and R. Eyraud. Polynomial identification in the limit of substitutable context-free languages. *Journal of Machine Learning Research*, 8:1725–1745, 2007.

[10] M. Wakatsuki and E. Tomita. A fast algorithm for checking the inclusion for very simple deterministic pushdown automata. *IEICE Transactions on Information and Systems*, E76-D(10):1224–1233, 1993.

[11] T. Kuriyama. *Learning Algorithm for Relation-Substitutable Context-Free Languages*. arXiv:1409.6247, originally posted in 2014 (v3: November 2014). `https://arxiv.org/abs/1409.6247`.

[12] R. Yoshinaka. General perspective on distributionally learnable classes. In *Proceedings of the 14th Meeting on the Mathematics of Language (MoL 14)*, pp. 87–98, 2015.

[13] A. Clark and R. Yoshinaka. Distributional learning of context-free and multiple context-free grammars. In *Topics in Grammatical Inference*, Springer, 2016.

[14] F. Coste, G. Garet, and J. Nicolas. Locally substitutable languages for enhanced inductive leaps. In *Proceedings of the Eleventh International Conference on Grammatical Inference*, PMLR 21:97–111, 2012.

[15] F. Coste and J. Nicolas. Learning local substitutable context-free languages from positive examples in polynomial time and data by reduction. In *Proceedings of the 14th International Conference on Grammatical Inference*, PMLR 93:155–168, 2019.

[16] R. Yoshinaka. Identification in the limit of k, l-substitutable context-free languages. In *Grammatical Inference: Algorithms and Applications – ICGI 2008*, LNCS 5278, pp. 266–279, Springer, 2008. doi:10.1007/978-3-540-88009-7_21.

[17] Y. Takada. Grammatical inference for even linear languages based on control sets. *Information Processing Letters*, 28(4):193–199, 1988. doi:10.1016/0020-0190(88)90208-6.

[18] Y. Takada. Learning formal languages based on control sets. In K. P. Jantke and S. Lange (Eds.), *Algorithmic Learning for Knowledge-Based Systems*, LNAI 961, pp. 316–339, Springer, 1995. doi:10.1007/3-540-60217-8_15.

[19] S. Ginsburg and E. H. Spanier. Finite-turn pushdown automata. *SIAM Journal on Control*, 4(3):429–453, 1966. doi:10.1137/0304034.

[20] F. Coste, D. Fredouille, C. Kermorvant, and C. de la Higuera. Introducing domain and typing bias in automata inference. In *ICGI 2004*, LNCS 3264, pp. 115–126, Springer, 2004. doi:10.1007/978-3-540-30195-0_11.

[21] A. Clark. Learning trees from strings: A strong learning algorithm for some context-free grammars. *Journal of Machine Learning Research*, 14:3537–3559, 2013.

[22] R. Eyraud, J. Heinz, and R. Yoshinaka. Efficiency in the identification in the limit learning paradigm. In J. Heinz and J. M. Sempere (Eds.), *Topics in Grammatical Inference*, Springer, 2016. doi:10.1007/978-3-662-48395-4_2.

[23] J.-M. Autebert, J. Berstel, and L. Boasson. Context-free languages and pushdown automata. In G. Rozenberg and A. Salomaa (Eds.), *Handbook of Formal Languages*, Vol. 1, pp. 111–174, Springer, 1997.

[24] S. Ginsburg and S. A. Greibach. Deterministic context-free languages. *Information and Control*, 9:620–648, 1966.

[25] A. Clark. Distributional learning of some context-free languages with a minimally adequate teacher. In *Proceedings of ICGI 2010*, LNCS 6339, pp. 24–37, Springer, 2010. doi:10.1007/978-3-642-15488-1_4.